웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 [202323035] 이름: [홍나린] 제출일: [2026-10-18]

서비스 이름 [한입 리뷰 (맛집 후기 게시판)]

기술 조합과 버전 [Vue 3.5.43(Vite 8.3.4) / Spring Boot 3.5.16(Java 17, JJWT 0.13.0, SQLite) / 실행 환경 macOS 15.7.9, Node.js v24.20.0, OpenJDK 17.0.20.1, 브라우저 Safari 26.6.1]

GitHub 저장소 URL [https://github.com/skfls0721/jwt-board-hanip-review]

소스 코드 기준 [main / c6e7e2f]

서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

[한입 리뷰는 먹어 본 식당과 메뉴에 대한 후기를 나누는 게시판입니다. 맛집 정보를 찾는 사람과 자신의 경험을 기록하려는 사람을 대상으로 합니다. 누구나 후기 목록과 상세 내용을 볼 수 있고, 로그인한 사용자만 후기를 작성할 수 있습니다. 수정과 삭제는 글을 쓴 사용자 본인만 할 수 있으며, 이 규칙을 화면의 버튼 표시뿐 아니라 서버에서도 검사합니다. 로그인은 JWT 로 처리하고 토큰은 브라우저 메모리에만 보관합니다.

기술 선택 이유: 프론트엔드는 다른 수업에서 Vue 를 사용해 본 경험이 있어, 이번 과제에서 더 깊이 학습해 보고 싶어 선택했습니다. 백엔드는 수업 시간에 Spring Boot 를 실습으로 다뤄 본 경험이 있어, 배운 내용을 이어서 JWT 인증과 작성자 권한을 직접 구현해 보려고 선택했습니다.

제공 템플릿에서 변경한 범위: 제공된 Vue 프론트와 Spring Boot 백엔드 시작 코드에서 인증 관련 TODO 를 구현했습니다. 백엔드는 AuthService.java 에서 JWT 발급, 토큰 검증, 작성자 권한 검사를 구현했고, BoardController.java 는 상태 확인 응답의 stack 필드를 제거한 것 외에는 제공 코드를 유지했습니다. 게시판 CRUD, 입력 검증, 오류 응답(Errors), 사용자 확인(Users), SQLite 저장 구조는 제공 코드를 그대로 사용했습니다. 프론트는 api.js 에서 토큰 보관·전송, 상태 코드별 오류 처리, 로그인, 작성자 비교(canEdit)를 구현했고, App.vue 에서 오류 처리와 화면 문구를 수정했습니다. 서비스 이름과 안내 문구는 맛집 후기 서비스에 맞게 index.html 과 App.vue 에서 바꿨습니다. 파일별 변경 내역은 2 절에 정리했습니다. 백엔드·프론트 폴더의 시작 안내 README 두 개는 완성본의 실행 방법과 구성에 맞게 다시 썼습니다.]

완료 여부는 완료·부분·미완료 중 하나로 기록하세요.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	[완료]	[BoardController.list/get, PostStore, 프론트 App.vue]	[제공 코드 유지. 비로그인 상태에서 조회 가능(T01)]
로그인과 JWT 발급	[완료]	[BoardController.login, Users.verify, AuthService.issueToken, 프론트 api.js login]	[login 과 Users 는 제공 코드, issueToken 과 프론트 연결은 구현. HS256, 기본 유효 시간 90 초, 로그인 실패 시 401(T02, T03)]

기능	완료 여부	관련 파일	비고 또는 제한
인증과 내 정보 조회	[완료]	[AuthService.currentUserId, BoardController.me]	[토큰 없음·변조·만료를 각각 401 로 구분(T04, T09, T10)]
본인 글 작성·수정·삭제	[완료]	[BoardController.create/update/delete, AuthService.assertOwner, 프론트 canEdit]	[컨트롤러는 제공 코드, assertOwner 와 canEdit 은 구현. 타인 글은 403, 작성자는 서버가 정함(T05~T08, T12)]
만료·로그아웃·오류 표시	[완료]	[프론트 api.js request/clearSession, App.vue logout, 서버 Errors]	[401 이면 로그인 상태 해제, 403 이면 유지(T10, T11)]
DB 데이터 유지	[완료]	[DatabaseConfig, schema.sql, PostStore]	[제공코드 유지, 서버 재시작 후 글 유지확인(T15)]

구조와 변경 내역

요청 흐름, 토큰의 이동, 변경한 파일의 역할을 연결해 설명하세요.

브라우저 → API 서버 → SQLite 의 연결 구조를 간단히 그리거나 글로 설명하세요. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하세요.

[전체 구조

브라우저 (Vue, http://127.0.0.1:4173)

| /api 요청을 Vite 프록시가 전달

▼

Spring Boot API 서버 (http://127.0.0.1:8000)

| JDBC

▼

SQLite 파일 (backend-spring/data/board.sqlite)

프론트는 npm run build 로 만든 화면을 npm run preview(4173 번)로 열고, /api 로 시작하는 요청은 vite.config.js 의 프록시가 8000 번 백엔드로 넘깁니다. 백엔드는 JdbcTemplate 으로 SQLite 파일에 게시글을 저장하며, 파일이 없으면 서버 시작 시 data/ 폴더와 테이블이 자동으로 만들어집니다. 사용자 계정(Alice, Bob)은 DB 가 아니라 Users 클래스가 서버 시작 시 메모리에 준비합니다.

로그인 요청의 흐름

1. 사용자가 로그인 폼을 제출하면 App.vue 가 api.js 의 login()을 호출합니다.
2. login()이 request()를 통해 POST /api/auth/login 에 이메일과 비밀번호를 JSON 으로 보냅니다.
3. 서버의 BoardController.login 이 입력 형식을 확인하고, Users.verify 로 비밀번호를 검증합니다. 실패하면 401 INVALID_CREDENTIALS 입니다.
4. 성공하면 AuthService.issueToken 이 HS256 JWT 를 만들고, accessToken, expiresIn, user 를 응답합니다.
5. login()이 응답의 토큰과 사용자 정보를 api.js 의 session 변수에 저장합니다.

게시글 수정 요청의 흐름

1. App.vue 의 save()가 api.update()를 호출합니다.
2. request()가 session 의 토큰으로 Authorization: Bearer <JWT> 헤더를 붙여 PUT /api/posts/{id}를 보냅니다.
3. BoardController.update 가 먼저 AuthService.currentUserId 로 토큰을 검증해 사용자 ID 를 얻습니다(실패 시 401).
4. 글을 조회하고(없으면 404), AuthService.assertOwner 로 글의 authorId 와 사용자 ID 를 비교합니다(다르면 403).
5. 입력 검사를 통과하면 PostStore.update 가 SQLite 를 수정하고, 수정된 글을 응답합니다.

토큰의 저장 위치와 전송 시점

토큰은 api.js 의 모듈 변수 session(브라우저 메모리)에만 저장하고 localStorage 와 sessionStorage 는 쓰지 않습니다. 로그인한 뒤에는 모든 /api 요청에 헤더가 붙고, 로그아웃·401 응답·새로고침이 일어나면 토큰이 사라집니다.]

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
[AuthService.java 생성자]	[설정 주입 없음]	[JWT_SECRET(64 자리 16 진수), issuer, audience, TTL 을 받아 키를 만들고, 형식 오류·예시 키는 서버 시작 시 거부] 출처: 수업 6 주차·7 주차 자료 및 AI(codex) 제안 후 반영	[BE-01]
[AuthService.issueToken]	[501 을 던짐]	[HS256 JWT 발급(sub, iss, aud, iat, exp)] 출처:6 주차 28 쪽 예제 참고	[BE-01]
[AuthService.currentUserId]	[1L 고정 반환]	[Bearer 헤더 읽기, 서명·알고리즘·만료·iss·aud·sub 검증, 사용자 존재 확인, 실패 시 401] 출처: 6 주차 35~40 쪽, 7 주차 50·53 쪽 참고	[BE-02]
[AuthService.validAudience]	[없음]	[aud 클레임이 문자열 또는 목록일때 설정값과 비교하는 보조함수] 출처: AI(codex) 제안 후 반영	[BE-02]
[AuthService.assertOwner]	[본문 비어있음]	[사용자 ID 와 authorId 가 다르면 403] 출처: 7 주차 65 쪽 참고	[BE-03]
[BoardController.health]	{"status":"ok","stack":"spring"}]	[stack 필드 제거] 출처: 본인 작성	[명세의 status: ok 객체에 맞춤]
[api.js 토큰 보관·헤더·401·login]	[TODO(헤더 {}, 401 미처리, 로그인 오류)]	[Bearer 헤더 첨부, 로그인 후 메모리 저장, 401 이면 clearSession] 출처: 6 주차 31·32·47 쪽 참고	[FE-01~03]
[api.js getErrorMessage, canEdit]	[없음, true 고정]	[상태·code 별 안내 문구, 로그인 사용자 ID 와 authorId 비교] 출처: AI(Codex) 초안 후 본인 수정	[FE-03, FE-04]
[api.js request 의 연결 실패·JSON 파싱 실패 처리]	[서버가 꺼지거나 JSON 이 아니면 영어 예외 노출]	[try/catch 두 곳 추가] 출처: AI(Claude) 제안 후 반영	[오류를 한국어 안내로 표시]

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
[App.vue 404 처리, logout 의 reset(), 비로그인 안내 문구]	[처리 없음]	[404 에서 선택 해제와 목록 갱신, 로그아웃 시 입력 폼 초기화, 비로그인 안내] 출처: AI(Claude) 제안 후 반영	[FE-03, FE-04]
[App.vue 서비스 이름·소개·배너·입력 예시 문구]	[시작 코드 문구]	[맛집 후기 서비스에 맞게 변경] 출처: 본인 작성	[서비스 주제 반영]
[index.html]	[제목 우리의 기록 | JWT 게시판 과제]	[제목을 한입 리뷰 | 맛집 후기 게시판으로 변경] 출처: 본인작성	[서비스 이름 반영]
[vite.config.js]	[프록시 설정(한 줄)]	[여러 줄로 정리(동작 변경 없음)] 출처: AI(Codex) 제안 후 반영	[가독성]
[그 외 모든 파일(Errors, Users, PostStore, DatabaseConfig, schema.sql, pom.xml, application.properties 등)]	[-]	[변경 없음]	[제공코드 유지]
tools/check_api.py	추가한 보조파일	자동 API 검사 도구 출처: 제공파일 그대로 사용	
tools/manual/*.sh, tools/manual/mktoken.py	추가한 보조파일	T01~T17 시험과 JWT 조작 시험(서명은 올바르고 내용만 바꾼 토큰, 헤더 형식 오류)을 실행하고 증거를 남기는 스크립트 출처: AI(Claude)가 작성	
README.md	추가한 보조파일	실행 방법과 구현 범위 출처: AI(Claude)가 초안 작성 후 수정	
docs/evidence/	추가한 보조파일	시험 출력 파일과 화면 캡처 출처: 직접 실행하여 생성	
backend-spring/README.md, frontend-vue/README.md	시작 코드의 실행 안내("로그인은 미완성입니다" 등)	완성본의 실행 방법·파일 구성 설명으로 교체 출처: AI(Claude)가 작성	옛 안내가 완성본과 맞지 않음

각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README 에서 연결하세요.

[운영체제와 버전: macOS 15.7.9, Node.js v24.20.0(npm 11.19.0), OpenJDK 17.0.20.1, 브라우저 Safari 26.6.1 (docs/evidence/environment.txt)

새 폴더 설치: 작업 폴더에서 node_modules, dist, target, .env, DB 파일을 뺀 제출용 폴더(jwt-board-submit)를 새로 만들고, 그 안에서 처음부터 설치·실행했습니다.

환경설정: 백엔드를 실행하는 터미널에서 JWT_SECRET 을 새 난수로 설정했습니다. 일반 시험은 TOKEN_TTL 기본값(90 초), 만료 시험은 TOKEN_TTL=3 으로 서버를 재시작했습니다.

백엔드 실행(backend-spring 폴더): export JWT_SECRET=$(openssl rand -hex 32) 후 ./mvnw spring-boot:run. 처음 실행에서 Maven 의존성을 내려받았고, "Started BoardApplication" 출력 후 GET /api/health 가 {"status":"ok"}, GET /api/posts 가 []을 반환했습니다.

프론트 빌드와 실행(frontend-vue 폴더): npm ci, npm run build, npm run preview. npm ci 와 npm run build 는 오류 없이 끝났고 로그는 docs/evidence/frontend-install.txt, docs/evidence/frontend-build.txt 에 남겼습니다.

SQLite 파일 경로: backend-spring/data/board.sqlite. 이 폴더와 파일이 없는 상태에서 서버를 처음 실행했을 때 자동으로 생성되는 것을 확인했습니다.]

[4173 번 접속과 API 연결: http://127.0.0.1:4173 에서 화면이 열렸고, 비로그인 상태의 글 목록 조회, Alice 와 Bob 로그인, 글 작성, 수정·삭제를 확인했습니다. 개발자 도구에서 /api 요청이 8000 번 백엔드로 전달되어 응답을 받는 것을 확인했습니다(T02, T05).

서버 재시작 후 데이터 유지: 글을 작성한 뒤 서버 프로세스를 종료하고 다시 실행했습니다. 재시작 전후의 서버 프로세스 번호가 달랐고(76791, 78233), 글 목록 전체가 동일했으며, 재시작 전에 작성한 글이 그대로 조회되었습니다(T15).

증거 위치: 환경 정보는 docs/evidence/environment.txt, 설치·빌드 결과는 docs/evidence/frontend-install.txt 와 frontend-build.txt, 4173 화면과 로그인·글 작성은 T02_login-screen.png, T05_post-created.png, T05_location-header.png, 서버 재시작 전후는 api-manual-t15-before.txt, api-manual-part4.txt, t15-posts-before.json, t15-posts-after.json, 새로고침 후 상태는 T16_after-refresh.png, 자동 검사는 api-check-final.txt(40 개 항목 통과)입니다. 자세한 재현 방법은 저장소 README 에 있습니다.

GitHub clone 후 실행 확인: GitHub 저장소(https://github.com/skfls0721/jwt-board-hanip-review)를 새 폴더에 clone 해 README 절차(./mvnw spring-boot:run, npm ci, npm run build, npm run preview)만으로 설치·빌드·실행했습니다. clone 직후에는 backend-spring/data 가 없었고 서버를 처음 실행하자 data/board.sqlite 가 자동으로 생성되었습니다. /api/health 가 {"status":"ok"}를 반환했고, python3 tools/check_api.py --mode final 이 40 개 항목 모두 통과했으며, 4173 번 화면에서 Alice 로그인과 글 작성을 확인했습니다. (확인 일시: 2026-10-17, 커밋 c6e7e2f) 이 clone 확인은 터미널에서 직접 확인했으며 별도 증거 파일로는 남기지 않았습니다.]

핵심 코드 설명

네 부분을 각각 설명하세요. 전체 파일을 붙이지 말고 핵심 코드 5~20 줄 정도를 발췌한 뒤 입력 → 처리 → 출력과 실패 조건을 자신의 말로 설명합니다. 성공·실패 시 HTTP 상태와 JSON code, 해당 분기를 만드는 코드도 연결해 적으세요. 줄 번호를 적는 경우 제출한 코드 기준인지 확인하세요.

코드 발췌는 제출한 파일에서 핵심 줄만 가져왔고, 생략한 부분은 …로 표시했습니다. 줄바꿈과 공백은 읽기 좋게 정리했습니다. 전체 코드는 저장소의 해당 파일에 있습니다.

3 1 로그인과 JWT 발급

파일 / 함수: backend-spring/.../BoardController.java 의 login (제공 코드 유지)

backend-spring/.../Users.java 의 verify (제공 코드 유지)

backend-spring/.../AuthService.java 의 생성자와 issueToken (인증 부분 구현)

코드 발췌

```
// BoardController.login (제공 코드)
var input = fields(body, Map.of("email", 254, "password", 128));
var user = users.verify(input.get("email"), input.get("password"));
if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");
return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);

// AuthService 생성자: 서버 키 확인
if (secret == null || !secret.matches("[0-9a-fA-F]{64}")
   || secret.equalsIgnoreCase("REPLACE_WITH_RANDOM_64_HEX_CHARACTERS")) { throw ...; }
this.key = Keys.hmacShaKeyFor(secretBytes);

// AuthService.issueToken
return Jwts.builder()
   .subject(Long.toString(user.id()))
   .issuer(issuer)
   .audience().add(audience).and()
   .issuedAt(Date.from(now))
   .expiration(Date.from(now.plusSeconds(ttl)))
   .signWith(key, Jwts.SIG.HS256)
   .compact();
```

동작과 설명

[**입력**: 브라우저가 POST /api/auth/login 으로 보내는 JSON(email, password)입니다.

입력 검사: fields 가 email 과 password 두 필드만 허용하고, 문자열인지, 공백만 있지 않은지, 길이가 이메일 254 자와 비밀번호 128 자 이하인지 확인합니다. 다른 필드를 붙이거나 형식이 틀리면 400 BAD_INPUT 입니다. 비밀번호는 앞뒤 공백을 자르지 않고 그대로 비교합니다.

계정 확인: Users.verify 가 이메일로 계정을 찾고, 입력한 비밀번호를 저장된 salt 로 다시 계산해 저장된 해시와 비교합니다. 비밀번호 원문은 저장하지 않고 해시만 보관합니다. 계정이 없는 경우에도 같은 방식으로 해시 계산을 한 번 하고 결과를 버리므로, 계정이 없을 때와 비밀번호가 틀릴 때의 응답이 같고 처리 시간도 비슷하도록 설계되어 있습니다. 그래서 어느 쪽이 틀렸는지 알아낼 수 없습니다.

토큰 만들기: 계정 확인에 성공한 경우에만 issueToken 이 실행됩니다. 토큰에는 사용자 ID(sub, 문자열), 발급자(iss), 사용 대상(aud), 발급 시각(iat), 만료 시각(exp = 지금 + TTL)을 넣고 서버 비밀 키로 HS256 서명합니다. 비밀번호나 해시는 토큰에 넣지 않습니다.

서버 키: JWT_SECRET 은 환경변수로 받습니다. 64 자리 16 진수를 32 바이트 키로 바꿔 쓰며, 값이 없거나(미설정이면 예시 키가 기본값), 형식이 틀리거나, 예시 키 그대로이면 서버가 시작되지 않습니다. 미설정, 예시 키 그대로, 빈 값, 16 진수 63 자리의 네 경우를 직접 시험했고, 모두 실행이 종료 코드 1 로 끝나며 “JWT_SECRET 은 32 바이트를 나타내는 64 자리 16 진수여야 합니다.” 오류가 출력되고 서버가 시작되지 않았습니다(api-key-rejection.txt).

출력: 성공하면 200 과 함께 accessToken, expiresIn(기본 90 초), user(id, name, email)를 돌려줍니다.

실패 조건과 응답: 없는 이메일, 틀린 비밀번호 -> http:401, code: INVALID_CREDENTIALS, 만들어지는 곳: login 의 user == null 분기

필수 값 누락, 길이 초과, 추가 필드, 깨진 JSON-> http:400, code: BAD_INPUT, 만들어지는 곳: fields, Errors 의 JSON 오류 처리

401 응답에는 Errors 클래스가 WWW-Authenticate: Bearer 헤더를 붙입니다.

근거 증거: 성공 응답과 expiresIn 은 docs/evidence/api-manual-part1.txt 의 T02, 틀린 비밀번호와 없는 이메일의 401 은 같은 파일의 T03, 발급된 토큰의 내용(HS256, sub·iss·aud·iat·exp, 유효 시간 90 초, 비밀번호·해시 미포함)은 docs/evidence/jwt-claims.txt 에 있습니다. JWT_SECRET 거부(미설정·예시 키·빈 값·형식 오류): docs/evidence/api-key-rejection.txt

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-vue/src/api.js 의 session, clearSession, authorizationHeaders, request, login

frontend-vue/src/App.vue 의 run, logout

코드 발췌

```
// api.js: 토큰은 메모리 변수에만 보관
let session = { accessToken: "", user: null };

function authorizationHeaders() {
  if (!session.accessToken) { return {}; }
  return { Authorization: `Bearer ${session.accessToken}` };
}

// login(): 응답의 토큰과 사용자 저장
session = { accessToken: result.accessToken, user: result.user };

// request(): 실패 응답 처리
if (!response.ok) {
  if (response.status === 401) { clearSession(); }
  const error = new Error(getErrorMessage(response.status, data?.code, data?.message));
  // ... (error.status, error.code 지정)
  throw error;
}

// App.vue logout()
clearSession(); user.value = null; reset();
```

동작과 응답 코드 설명

[**입력**: 로그인 폼의 이메일과 비밀번호, 그리고 이후 모든 API 호출입니다.

저장 위치: 로그인 응답의 accessToken 과 user 는 api.js 의 모듈 변수 session(브라우저 메모리)에만 저장합니다. localStorage 와 sessionStorage 는 사용하지 않습니다(frontend-vue/src 전체를 검색해 확인했습니다). 그래서 새로고침하거나 탭을 닫으면 로그인 상태가 사라지며, 이는 과제 안내에서 정상으로 정한 동작입니다.

전송: 모든 API 요청은 request()를 거치고, 그 안에서 authorizationHeaders()가 토큰이 있을 때만 Authorization: Bearer <JWT> 헤더를 붙입니다. 토큰이 없으면 헤더를 붙이지 않으므로 Bearer undefined 같은 값이 나가지 않습니다. 목록·상세 조회도 같은 request()를 쓰지만 로그인 전에는 헤더가 없고, 서버가 공개 API 로 처리하므로 토큰 없이 조회됩니다.

로그아웃: clearSession()으로 메모리의 토큰과 사용자를 비우고, 화면의 로그인 표시와 입력 폼도 함께 지웁니다(테스트 증거 8). 이후 요청에는 헤더가 붙지 않아 서버가 401 TOKEN_REQUIRED 로 응답합니다. 이 로그아웃은 이미 복사된 토큰을 서버에서 무효화하지 않으며, 토큰은 만료 시각까지 유효합니다.

응답 코드별 처리 (request 와 App.vue 의 run 이 함께 처리): 200,201-응답 결과로 목록·상세·사용자 상태를 갱신, 204- JSON 을 파싱하지 않고 성공 처리(삭제 후 목록 갱신), 400- 서버 메시지를 안내. 성공했을 때만 입력 폼을 비우므로 입력값이 남음(테스트 증거 10), 401- clearSession()으로 토큰·사용자 해제, 화면도 로그아웃 상태. 만료·토큰 없음·변조·로그인 실패를 code 별 문구로 안내, 403- 권한 없음을 안내. 토큰은 유효하므로 로그아웃하지 않음(테스트 증거 6), 404- 글 없음을 안내하고 선택과 폼을 지운 뒤 목록을 다시 불러옴(화면 확인: 테스트 증거 7), 서버 연결 실패- 프론트 서버까지 꺼져 fetch 가 실패하면 "서버에 연결할 수 없습니다" 안내. 프론트 서버는 켜져 있고 백엔드만 꺼진 경우에는 프록시가 502 를 돌려주어 "요청 처리에 실패했습니다. (HTTP 502)"가 표시됨(테스트 증거 9)

실패 조건: 비밀번호가 틀리면 login 이 오류를 던지므로 session 이 바뀌지 않고 로그인 화면으로 넘어가지 않습니다.

근거 증거: 로그인 후 화면 T02_login-screen.png, 요청에 토큰이 첨부된 모습(토큰 일부 가림) T10_expired-headers.png, 글 등록 후 201 응답과 Location 헤더 T05_location-header.png, 로그아웃 뒤 요청에 Authorization 이 없는 화면 T11_logout.png 와 T11_no-auth-header.png, 새로고침 후 로그인 해제 T16_after-refresh.png, 400 화면 동작 FE03_400-screen.png , 401 처리와 안내 T10_expired-screen.png, 403 안내와 로그인 유지 T07_403-screen.png, 404 화면 동작 FE03_404-screen.png, 로그아웃 시 폼 초기화 FE04_logout-form-before.png 와 FE04_logout-form-after.png, 서버 꺼짐 안내 FE03_backend-down.png 와 FE03_front-down.png, 브라우저 저장소 미사용 검색 결과 no-browser-storage.txt 입니다.

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: backend-spring/.../AuthService.java 의 currentUserId, validAudience

backend-spring/.../Errors.java 의 api (401 응답 헤더, 제공 코드 유지)

backend-spring/.../BoardController.java 의 me, create, update, delete (검증 결과를 받아 쓰는 쪽, 제공 코드 유지)

코드 발췌

```
// AuthService.currentUserId
String header = request.getHeader("Authorization");
if (header == null || !header.matches("(?i)^Bearer\\s+\\S+$")) {
  throw new ApiError(401, "TOKEN_REQUIRED", "유효한 Bearer 토큰이 필요합니다.");
}
// ... (Bearer 뒤의 토큰 문자열을 token 에 저장)
try {
  var jws = Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
  if (!"HS256".equals(jws.getHeader().getAlgorithm())) { ... }
  // ... exp·iss·aud(validAudience)·sub 형식 검사, 이상이면 401 INVALID_TOKEN
  // ... users.find(userId) == null 이면 401 INVALID_TOKEN
  return userId;
} catch (ExpiredJwtException e) {
  throw new ApiError(401, "TOKEN_EXPIRED", "JWT 가 만료되었습니다.");
} catch (JwtException | IllegalArgumentException e) {
  throw new ApiError(401, "INVALID_TOKEN", "JWT 검증에 실패했습니다.");
}
```

동작과 응답 코드 설명

입력: 요청의 Authorization 헤더입니다. 보호 API(/me, 글 등록·수정·삭제)가 실행될 때마다 컨트롤러가 가장 먼저 currentUserId 를 호출합니다.

헤더 확인: 헤더가 없거나 Bearer 토큰 모양이 아니면 토큰이 없는 것으로 보고 401 TOKEN_REQUIRED 로 끝냅니다.

서명과 만료 검증: parseSignedClaims 가 서버 비밀 키로 서명을 검증하고 만료 시각도 확인합니다. 단순히 토큰 안의 내용을 읽기만 하는 decode 가 아니라, 서명이 맞아야만 내용을 돌려주는 verify 입니다. 내용을 바꾸면 기존 서명과 맞지 않아 예외가 발생합니다.

추가 확인: 라이브러리 검증을 통과한 뒤에도 알고리즘이 HS256 인지 exp 가 있는지 발급자(iss)가 서버 설정과 같은지 사용 대상(aud)에 서버 설정값이 들어 있는지(validAudience 가 aud 가 문자열일 때와 목록일 때를 모두 비교합니다. 실제 발급된 토큰은 aud 가 ['board-api'] 목록이었습니다.) sub 가 양의 정수 모양의 문자열인지 코드가 직접 확인합니다.

사용자 확인: sub 를 사용자 ID 로 바꿔 실제 계정이 있는지 users.find 로 확인합니다. 토큰이 정상이어도 계정이 없으면 거부합니다.

출력: 모두 통과하면 검증된 사용자 ID 를 돌려줍니다. 컨트롤러는 이 값만 사용하고, 요청 본문에 userId 나 authorId 가 있어도 쓰지 않습니다. 본문의 추가 필드는 입력 검사에서 400 으로 거부됩니다

실패 조건과 응답:

헤더 없음, Bearer 토큰 모양이 아님- http: 401 code: TOKEN_REQUIRED 만들어지는 곳: 정규식 검사

서명 변조, 형식이 깨진 토큰 - http: 401 code: INVALID_TOKEN 만들어지는 곳: JwtException 처리

알고리즘·exp·iss·aud·sub 이상- http: 401 code: INVALID_TOKEN 만들어지는 곳: 클레임 검사

계정이 없는 사용자- http: 401 code: INVALID_TOKEN 만들어지는 곳: users.find 검사

정상 서명의 만료된 토큰- http: 401 code: TOKEN_EXPIRED 만들어지는 곳: ExpiredJwtException 처리

모든 401 응답에는 Errors.api 가 WWW-Authenticate: Bearer 헤더를 붙입니다. 만료와 변조를 구분하는 이유는, 만료는 "다시 로그인하세요", 변조는 "인증 정보가 유효하지 않습니다"처럼 화면에서 다른 안내를 보여 주기 위해서입니다.

검증 순서: 인증이 글 조회와 권한 검사보다 먼저 실행됩니다. 그래서 만료된 Bob 의 토큰으로 Alice 글을 수정해도 403 이 아니라 401 TOKEN_EXPIRED 가 나옵니다.

근거 증거: 토큰 없이 /me, 글 등록: api-manual-part1.txt 의 T04 (401 TOKEN_REQUIRED, WWW-Authenticate 있음)

서명 변조(첫 글자 변경)로 GET /me, POST, PUT, DELETE: api-manual-part2.txt 의 T09 (모두 401 INVALID_TOKEN, 이후 원문과 글 개수 유지)

정상 서명의 만료 토큰으로 GET /me, POST, PUT, DELETE 와 Bob 의 만료 토큰으로 Alice 글 수정: api-manual-part4.txt 의 T10 (모두 401 TOKEN_EXPIRED)

화면의 만료 처리: T10_expired-screen.png, T10_expired-headers.png, T10_expired-response.png

정상 토큰: api-manual-part1.txt 의 T05 (/me 가 Alice 를 반환)

토큰 내용(HS256, sub·iss·aud·iat·exp): jwt-claims.txt

추가 조작 시험: api-manual-part5.txt (대조군 포함 31 건 통과)

추가 검증 (JWT 조작 시험): 본인의 로컬 시험용 키로 서버를 실행하고, 같은 키로 서명한 토큰의 내용만 바꿔서 시험했습니다. 시험용 키로 만든 정상 토큰은 200 으로 통과해 토큰 생성 방식이 맞음을 확인했습니다. Authorization 헤더 형식 오류(Basic, 토큰 없음, 토큰 2 개)는 401 TOKEN_REQUIRED, JWT 모양이 아닌 문자열·HS512 서명·alg: none·iss/aud 불일치·iss/exp/sub 누락·존재하지 않는 사용자(sub=999)·문자 sub·앞에 0 이 붙은 sub·숫자 타입 sub 는 모두 401 INVALID_TOKEN 이었고, GET /api/me 와 PUT /api/posts/{id} 두 요청 모두에서 같았습니다. 모든 401 응답에 WWW-Authenticate: Bearer 헤더가 있었고, 시험 뒤 글 원문과 글 개수는 그대로였습니다. 다만 이 시험은 거부 결과만 보여 주므로, 라이브러리 검증에서 걸렸는지 서버 코드의 추가 확인에서 걸렸는지는 구분하지 않았습니다. 시험용 키는 저장소 밖의 임시 파일에 두었다가 시험 뒤 삭제했습니다.

3 4 작성자 권한 검사

파일 / 함수: backend-spring/.../AuthService.java 의 assertOwner (인증 부분 구현)

backend-spring/.../BoardController.java 의 create, update, delete (제공 코드 유지)

backend-spring/.../PostStore.java 의 update (제공 코드 유지)

frontend-vue/src/api.js 의 canEdit, frontend-vue/src/App.vue 의 수정·삭제 버튼 (화면 표시)

코드 발췌

```
// AuthService.assertOwner
if (userId != authorId) {
  throw new ApiError(403, "FORBIDDEN", "게시글 작성자만 수정하거나 삭제할 수 있습니다.");
}


// BoardController.update (제공 코드)
long user = auth.currentUserId(req), pid = postId(id);
var p = existing(pid);
auth.assertOwner(user, p.authorId());
var input = fields(body, Map.of("title", 100, "content", 5000));
```

return store.update(pid, input.get("title"), input.get("content"));

// PostStore.update (제공 코드): 바뀌는 열은 title, content, updated_at 뿐

"UPDATE posts SET title=?,content=?,updated_at=? WHERE id=?"

// api.js canEdit / App.vue: 내 글에만 수정·삭제 버튼 표시

return Boolean(session.user && post && Number(session.user.id) === Number(post.authorId));

 ...

동작과 응답 코드 설명

[**비교 대상**: assertOwner 는 두 값을 비교합니다. 하나는 3.3 의 currentUserId 가 토큰에서 얻은 **현재 사용자 ID** 이고, 다른 하나는 DB 에 저장된 글의 **작성자 ID**(p.authorId())입니다. 요청 본문이나 화면이 보낸 값은 비교에 쓰지 않습니다.

처리 순서 (수정·삭제 공통): ① 인증(실패 시 401) → ② 글 존재 확인(없으면 404) → ③ 작성자 비교(다르면 403) → ④ 입력 검사(틀리면 400) → ⑤ DB 변경. 작성자 비교가 입력 검사보다 먼저이므로, 같은 공백 제목이라도 작성자인 Alice 는 400, 다른 사람인 Bob 은 403 을 받습니다. 또한 403 이면 DB 변경 단계에 도달하지 않으므로 데이터가 바뀌지 않습니다.

401 과 403 의 차이: 401 은 "누구인지 확인할 수 없다"(토큰 없음·변조·만료), 403 은 "누구인지는 확인했지만 그 글을 바꿀 권한이 없다"입니다. 403 은 로그인 자체는 유효하므로 화면에서도 로그아웃시키지 않습니다.

작성자 위조 방지: 글을 등록할 때 작성자는 서버가 currentUserId 로 정합니다. 요청에 authorId, userId, id 같은 추가 필드를 넣으면 fields 가 400 BAD_INPUT 으로 거부합니다. 수정 SQL 은 title, content, updated_at 만 바꾸므로 작성자와 생성 시각은 수정해도 유지됩니다.

화면 버튼과 서버 검사의 관계: canEdit 은 로그인 사용자 ID 와 글의 authorId 가 같을 때만 수정·삭제 버튼을 보여 줍니다. 이것은 사용자가 할 수 없는 작업을 눈에 띄지 않게 하는 편의 기능입니다. 버튼을 숨기거나 개발자 도구로 화면을 바꾸더라도 서버가 assertOwner 로 다시 검사하므로, 직접 요청을 보내도 같은 결과(403)가 나옵니다. 화면의 수정 폼을 통해 타인 글 수정을 요청하는 경우에도 서버가 403 으로 거부하고 글이 바뀌지 않음을 화면에서 확인했습니다(테스트 증거 6).

출력: 작성자가 맞으면 수정은 200 과 수정된 글, 삭제는 204(본문 없음)입니다.

실패 조건과 응답:

토큰 없음·변조·만료– http: 401 code: TOKEN_REQUIRED / INVALID_TOKEN / TOKEN_EXPIRED 만들어지는 곳: currentUserId

글이 없음– http: 404 code: NOT_FOUND 만들어지는 곳: existing

다른 사람의 글을 수정·삭제– http: 403 code: FORBIDDEN 만들어지는 곳: assertOwner

입력이 틀림, 추가 필드– http: 400 code: BAD_INPUT 만들어지는 곳: fields

글 ID 형식 오류– http: 400 code: BAD_INPUT 만들어지는 곳: postId

근거 증거:Alice 글 등록, authorId 1: api-manual-part1.txt 의 T05

본인 글 수정 200(작성자·createdAt 유지, updatedAt 만 변경)과 삭제 204: 같은 파일의 T06

Bob 이 Alice 글 수정·삭제 시도 403 과 이후 원문 유지: 같은 파일의 T07, T08

작성자 위조 시도(authorId, userId, id) 400, 글 개수 유지: api-manual-part2.txt 의 T12

같은 공백 제목에 Alice 는 400, Bob 은 403, 없는 글은 정상 토큰이면 404·토큰 없으면 401: api-manual-part3.txt 의 오류 겹침 항목

Bob 의 본인 글 등록·수정·삭제, 토큰 없는 수정·삭제 401: 같은 파일의 T17

만료된 Bob 의 토큰으로 Alice 글 수정 시 401: api-manual-part4.txt 의 T10

화면 버튼: T07-08_bob-no-buttons.png(Alice 글에 버튼 없음), T07-08_bob-own-buttons.png(Bob 글에 버튼 있음)

화면 403 안내와 글 불변: T07_403-screen.png, t07-403-after.txt

.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: [macOS 15.7.9 / Node.js v24.20.0(npm 11.19.0) / OpenJDK 17.0.20.1 / Safari 26.6.1 / 백엔드 ./mvnw spring-boot:run, 프론트 npm ci, npm run build, npm run preview / 시험 tools/manual/part1~5.sh(curl), tools/check_api.py]

로컬 배포 확인: [위 "로컬 실행 기록"에 정리했습니다. 작업 폴더의 복사본에서 처음부터 설치·빌드·실행했고, GitHub 에 올린 저장소를 새 폴더에 clone 해 README 절차로 다시 실행해 확인했습니다(로컬 실행 기록의 마지막 단락).]

테스트 일시 및 코드 버전: [2026-10-15~18 / 커밋 c6e7e2f. 시험을 시작한 뒤 서비스 소스 코드(frontend-vue/src, backend-spring/src)는 바꾸지 않았고, 이후 변경은 README 와 시험 스크립트의 증거 폴더 경로 수정뿐입니다]

토큰 TTL: [일반 시험 90 초(기본값), 만료 시험 3 초(TOKEN_TTL=3 으로 서버 재시작)]

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

증거 파일 이름은 docs/evidence/ 기준이고, 표에서는 이렇게 줄여 씁니다. P1=api-manual-part1.txt, P2=api-manual-part2.txt, P3=api-manual-part3.txt, P4=api-manual-part4.txt, P5=api-manual-part5.txt.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	[토큰 없이 GET /api/posts, GET /api/posts/{id}]	[200]	[200 / 해당 없음]	[PASS]	[P1(T01)]
T02	[Alice 로 POST /api/auth/login, 화면 로그인]	[200, JWT 와 사용자 정보, 화면 표시]	[200 / 해당 없음. accessToken, expiresIn 90, user 반환. 화면에 “Alice 님 로그인 중”]	[PASS]	[P1(T02), T02_login-screen.png, jwt-claims.txt]
T03	[틀린 비밀번호, 없는 이메일로 각각 로그인]	[401 INVALID_CREDENTIALS, 토큰 미발급]	[401 / INVALID_CREDENTIALS (두 경우 모두, 응답에 accessToken 없음)]	[PASS]	[P1(T03)]
T04	[토큰 없이 GET /api/me, POST /api/posts]	[각각 401 TOKEN_REQUIRED]	[401 / TOKEN_REQUIRED (두 요청 모두)]	[PASS]	[P1(T04)]

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T05	[Alice 토큰으로 글 작성, GET /api/me]	[201, authorId 1, /me 는 Alice]	[201 / 해당 없음(Location: /api/posts/2, authorId 1), /me 200 Alice. 화면에서도 201 확인]	[PASS]	[P1(T05), T05_post-created.png, T05_location-header.png]
T06	[Alice 가 본인 글 수정, 다른 본인 글 삭제]	[200, 204, DB 반영]	[수정 200(updatedAt 만 변경, 작성자·createdAt 유지), 삭제 204(본문 없음), 삭제 뒤 조회 404]	[PASS]	[P1(T06)]
T07	[Bob 의 정상 토큰으로 Alice 글 수정(curl). 화면에서는 Alice 로 수정 폼을 연 뒤 로그아웃 없이 Bob 으로 로그인해 수정 저장]	[403, 원문 유지]	[403 / FORBIDDEN, 재조회하니 원문 그대로. 화면에서 Alice 글에 수정·삭제 버튼 없음. 폼으로 저장을 시도하면 "권한이 없습니다…" 안내가 뜨고 로그인 유지]	[PASS]	[테스트 증거 2, 테스트 증거 6,P1(T07), T07-08_bob-no-buttons.png, T07-08_bob-own-buttons.png, T07_403-screen.png]
T08	[Bob 의 정상 토큰으로 Alice 글 삭제]	[403, 글 유지]	[403 / FORBIDDEN, 재조회하니 글 유지]	[PASS]	[P1(T08)]
T09	[서명 첫 글자를 바꾼 토큰으로 GET /me, POST, PUT, DELETE]	[401 INVALID_TOKEN, 데이터 변경 없음]	[4 건 모두 401 / INVALID_TOKEN, 원문과 글 개수 유지. 추가 조작 시험 31 건도 모두 예상대로(헤더 형식 오류·알고리즘·클레임·sub)]	[PASS]	[P2(T09), P5]
T10	[TOKEN_TTL=3 으로 발급, 6 초 뒤 GET /me, POST, PUT, DELETE, Bob 의 만료 토큰으로 Alice 글 수정]	[401 TOKEN_EXPIRED, 화면 로그인 해제와 안내]	[5 건 모두 401 / TOKEN_EXPIRED, 원문과 글 개수 유지. 화면에 "로그인 시간이 만료되었습니다. 다시 로그인해 주세요."와 "로그아웃 상태"]	[PASS]	[P4(T10), T10_expired-screen.png, T10_expired-headers.png, T10_expired-response.png]
T11	[로그인 후 로그아웃, 다음 /me 요청]	[요청에서 토큰이 빠지고 401 TOKEN_REQUIRED]	[로그아웃 뒤 요청 헤더에 Authorization 없음, 401 / TOKEN_REQUIRED. 화면에 "로그인이 필요합니다. 다시 로그인해 주세요."]	[PASS]	[T11_logout.png, T11_no-auth-header.png, P1(T04)]

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T12	[Bob 이 authorId/userId/id 를 붙여 글 등록, Alice 가 authorId 를 붙여 수정]	[400 BAD_INPUT, 위조 글 미생성]	[4 건 모두 400 / BAD_INPUT, 글 개수와 원문 유지]	[PASS]	[P2(T12)]
T13	[본인 토큰으로 빈 제목, 공백 본문, 제목 101 자, 내용 5001 자를 등록·수정으로 각각 전송]	[400 BAD_INPUT, DB 변경 없음]	[8 건 모두 400 / BAD_INPUT, 글 개수와 원문 유지, 화면에서도 공백 제목 등록 시 "입력 오류" 안내가 표시되고 입력값이 유지됨]	[PASS]	[P2(T13), 테스트 증거 10]
T14	[없는 ID(999999)로 조회·수정·삭제(수정·삭제는 정상 토큰), 잘못된 ID(abc, 0, -1, 1.5)]	[없는 글 404, 잘못된 ID 400]	[없는 글 3 건 404 / NOT_FOUND, 잘못된 ID 조회·수정·삭제 400 / BAD_INPUT]	[PASS]	[P3(T14), 테스트 증거 7]
T15	[글 작성 뒤 서버 프로세스를 종료하고 다시 실행]	[기존 글과 작성자·내용유지]	[서버 PID 가 76791 에서 78233 으로 바뀜. 재시작 전후 글 목록 전체가 동일, 재시작 전에 쓴 글이 200 으로 조회됨]	[PASS]	[api-manual-t15-before.txt, P4(T15), t15-posts-before.json, t15-posts-after.json]
T16	[로그인 후 페이지 새로고침]	[공개 글 유지, 메모리 로그인 상태 해제]	[새로고침 뒤 "로그아웃 상태", 글 목록 유지]	[PASS]	[T16_after-refresh.png]
T17	[토큰 없는 수정·삭제, Bob 의 정상 작성·본인 수정·삭제]	[무토큰 401, Bob 본인 작업 성공]	[무토큰 수정·삭제 401 / TOKEN_REQUIRED(원문 유지). Bob 등록 201(Location: /api/posts/7), 수정 200, 삭제 204, 삭제 뒤 조회 404. Alice 글은 그대로]	[PASS]	[P3(T17)]

T15 시험을 위해 서버를 다시 켤 때 TOKEN_TTL=3 으로 켰고, 이어서 T10 만료 시험을 했습니다. 글 데이터는 TTL 과 무관하게 DB 파일에 남습니다.

T09~T13 시험 스크립트(P2)는 출력 문구의 오류(??)를 고친 뒤 한 번 더 실행해 그 결과를 최종 증거로 사용했습니다. 앞 실행과 결과는 같고, 글 개수만 시험 시점의 DB 상태에 따라 다릅니다.

T14 에서 GET /api/posts/1.5 응답에는 Content-Disposition 헤더가 같이 붙었습니다. 주소 끝 .5 를 파일 확장자로 보는 Spring 의 기본 동작이며, 상태 코드와 JSON code 는 명세대로 400 BAD_INPUT 이었습니다.

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	[200 / 해당 없음]	[P1(T01, T02, T06)]
새 글 등록	**201 및 Location**	[201, Location: /api/posts/2]	[P1(T05), T05_location-header.png]
본인 글 삭제	**204 및 빈 본문**	[204, 본문 없음]	[P1(T06), P3(T17)]
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	[400 / BAD_INPUT]	[P2(T12, T13), P3(T14), 테스트 증거 10]
잘못된 계정	**401 INVALID_CREDENTIALS**	[401 / INVALID_CREDENTIALS]	[P1(T03)]
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	[401 / TOKEN_REQUIRED]	[P1(T04), P5(헤더 형식)]
변조·클레임 오류	**401 INVALID_TOKEN**	[401 / INVALID_TOKEN]	[P2(T09), P5]
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	[401 / TOKEN_EXPIRED]	[P4(T10), T10_expired-response.png]
다른 작성자의 변경	**403 FORBIDDEN**	[403 / FORBIDDEN]	[P1(T07, T08),테스트증거 6, t07-403-after.txt]
없는 글	**404 NOT_FOUND**	[404 / NOT_FOUND]	[P3(T14)]

보호 API 401 의 WWW-Authenticate: Bearer: P1~P5 에서 확인한 모든 401 응답(로그인 실패 포함)에 WWW-Authenticate: Bearer 가 있었습니다. 400, 403, 404 응답에는 없었습니다. 화면에서도 T10_expired-headers.png 로 같은 헤더를 확인했습니다.

401 과 403 에서 화면의 차이: 401 을 받으면 토큰과 사용자가 해제되어 로그인 카드가 "로그아웃 상태"가 되고 만료·토큰 없음 등 code 에 맞는 안내가 표시됩니다(T10_expired-screen.png, T11_logout.png). 403 을 받으면 안내만 표시되고 로그인은 유지됩니다. 타인 글에는 수정·삭제 버튼이 보이지 않아 정상적인 화면 조작으로는 403 요청을 보낼 수 없었습니다. 그래서 Alice 로 로그인해 본인 글의 수정 폼을 연 뒤, 로그아웃하지 않고 Bob 으로 로그인하면 수정 폼이 그대로 남는 점을 이용해, 그 폼으로 Alice 글의 수정 저장을 요청했습니다. 그 결과 상단에 "권한이 없습니다. 본인이 작성한 게시글만 수정하거나 삭제할 수 있습니다."가 표시되었고,

로그인 카드는 “Bob 님 로그인 중”으로 유지되었으며(자동 로그아웃 없음), 개발자 도구에서 PUT 요청의 상태가 403 이었습니다(T07_403-screen.png, 증거 6). 이어서 글 목록을 조회해, Bob 의 수정 시도에서 덧붙인 문구 “(Bob 수정 시도)”가 Alice 글 제목에 들어가지 않았음을 확인했습니다. 시험 직후의 터미널 출력은 파일로 저장하지 못해, 같은 DB 를 이후에 다시 조회한 기록을 t07-403-after.txt 로 남겼습니다. 이 글의 수정 시각(updated_at)이 등록 시각과 다른 것은 같은 시험 중 Alice 계정으로 직접 저장한 시각이며, Bob 의 요청(403)으로 바뀐 것이 아닙니다.

실패한 수정·삭제 뒤 데이터 불변: T07, T08(P1), T09(P2), T10(P4), T12·T13(P2), 겹침 사례와 T17(P3)에서 실패 요청 뒤마다 공개 조회로 원문과 글 개수가 그대로임을 확인했습니다.

토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404: P3 에서 같은 없는 글 999999 에 토큰 없이 수정하면 401 TOKEN_REQUIRED, 정상 토큰으로 수정하면 404 NOT_FOUND 였습니다. 같은 파일에서 Bob 이 Alice 글에 공백 제목을 보내면 403, Alice 가 자기 글에 보내면 400 이었습니다. 만료된 Bob 의 토큰은 권한 검사 전에 401 TOKEN_EXPIRED 였습니다(P4).

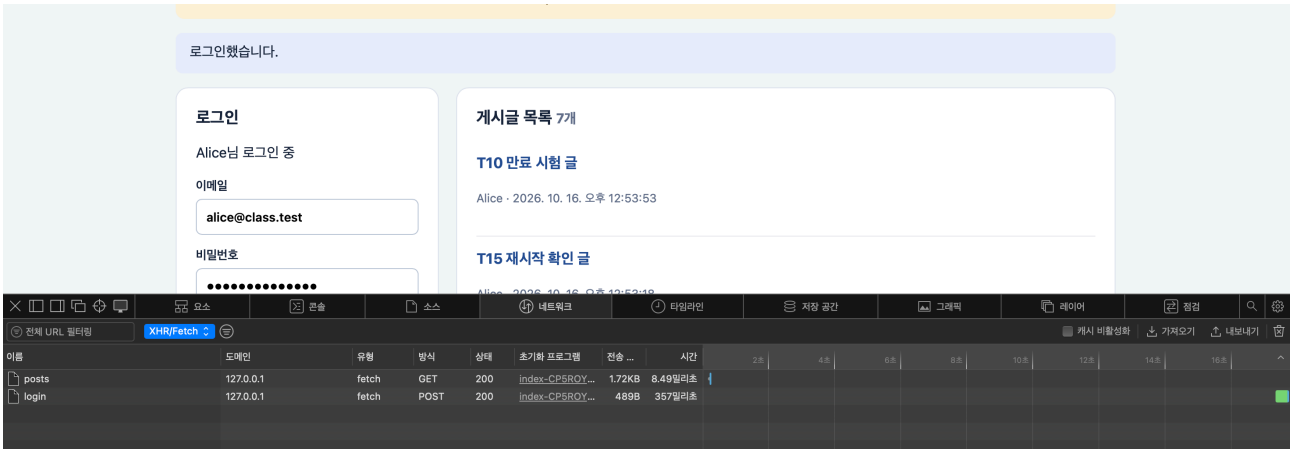
테스트 증거

캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

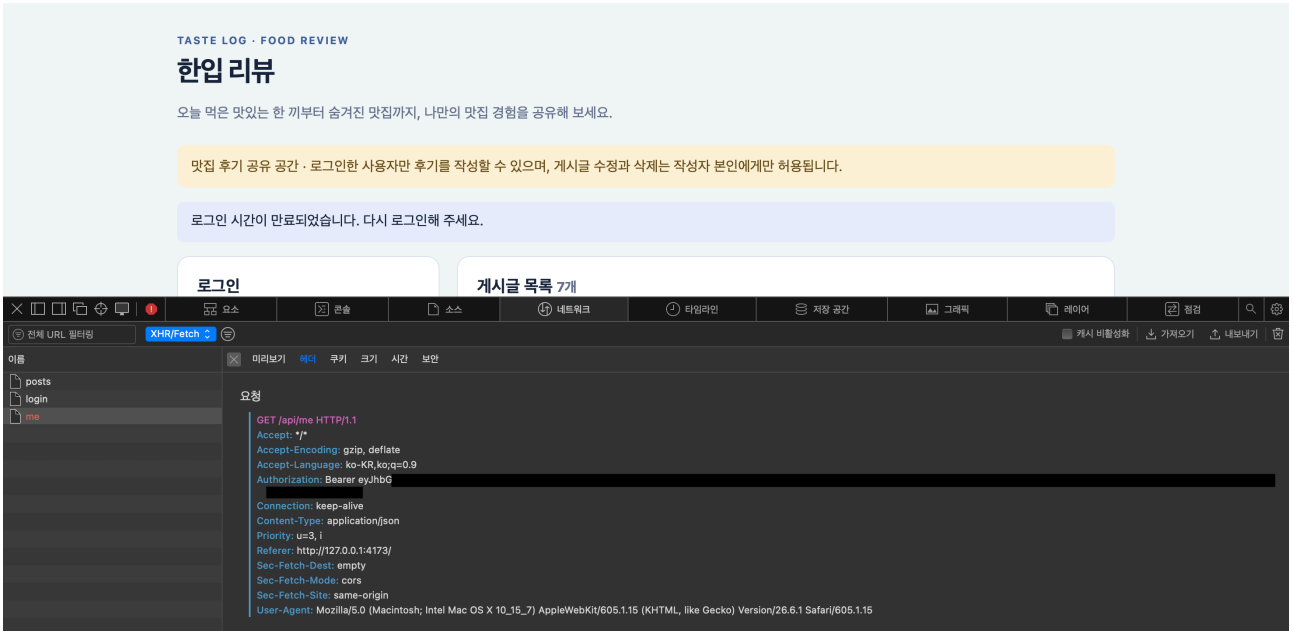
로그인과 토큰 전송

[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]

[T02_login-screen.png(로그인 성공 화면), T10_expired-headers.png(Authorization: Bearer 헤더, 토큰 일부 가림), jwt-claims.txt(발급된 토큰의 내용)]



T02_login-screen.png(로그인 성공 화면)

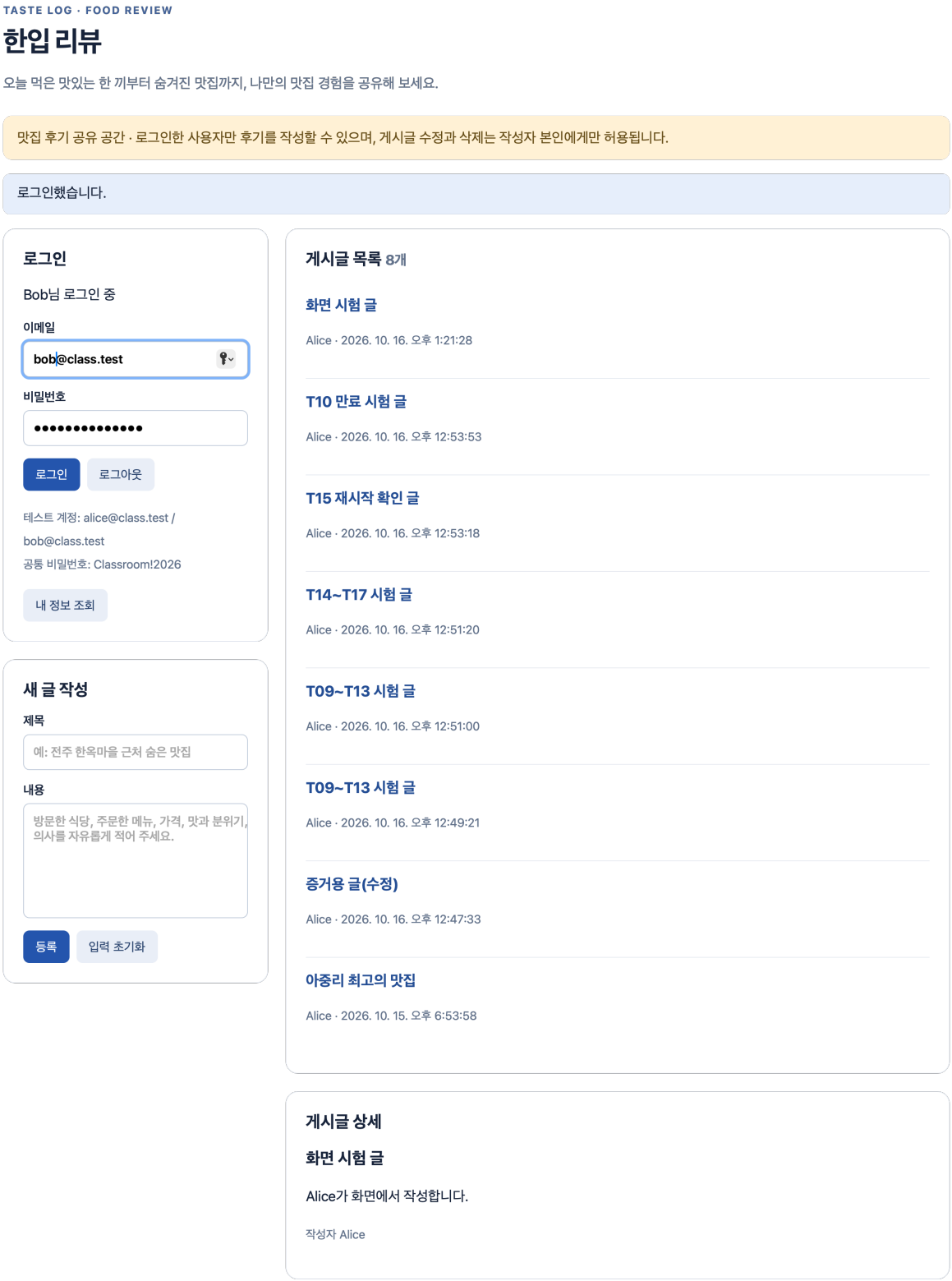


T10_expired-headers.png(Authorization: Bearer 헤더, 토큰 일부 가림)

다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

[P1(T07, T08)의 서버 응답, 화면은 T07-08_bob-no-buttons.png]



T07-08_bob-no-buttons.png

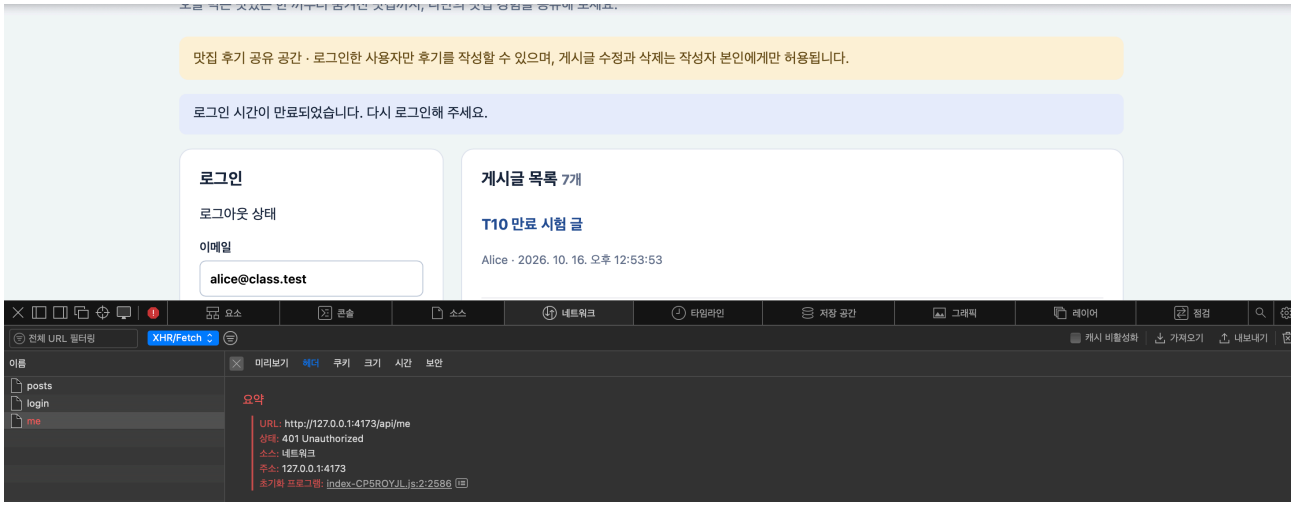
본인 작성글의 삭제와 수정 (T07-08_bob-own-buttons.png, 서버 응답은 P1(T07, T08)



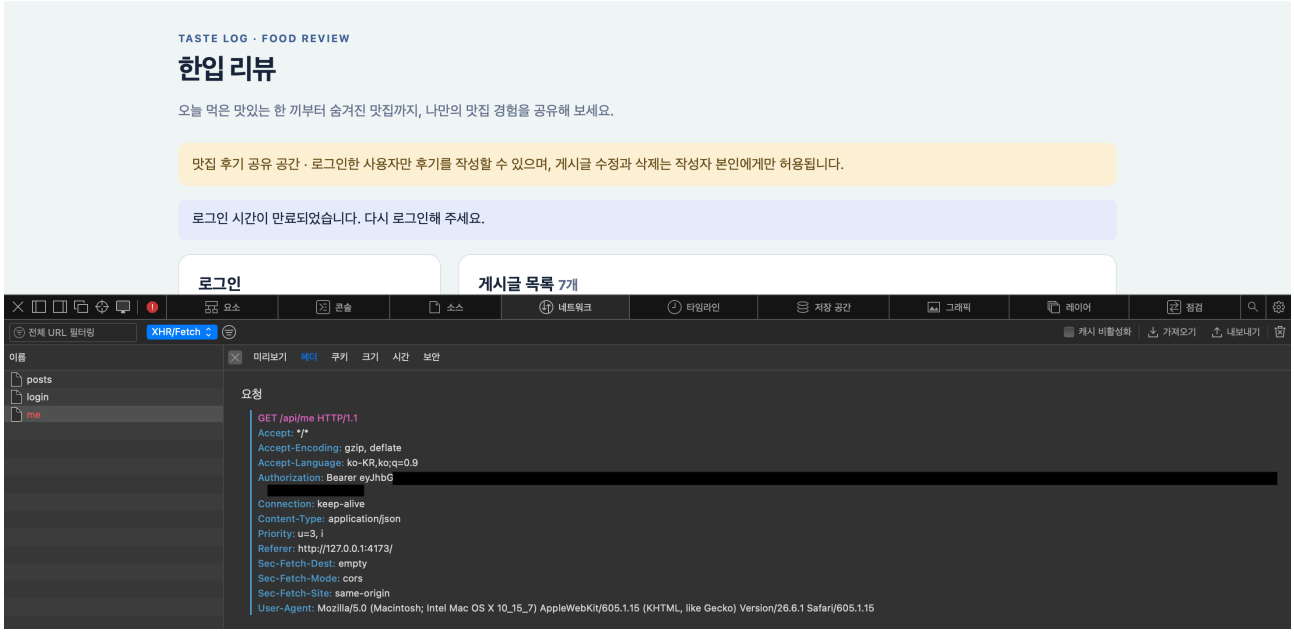
토큰 변조와 만료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]

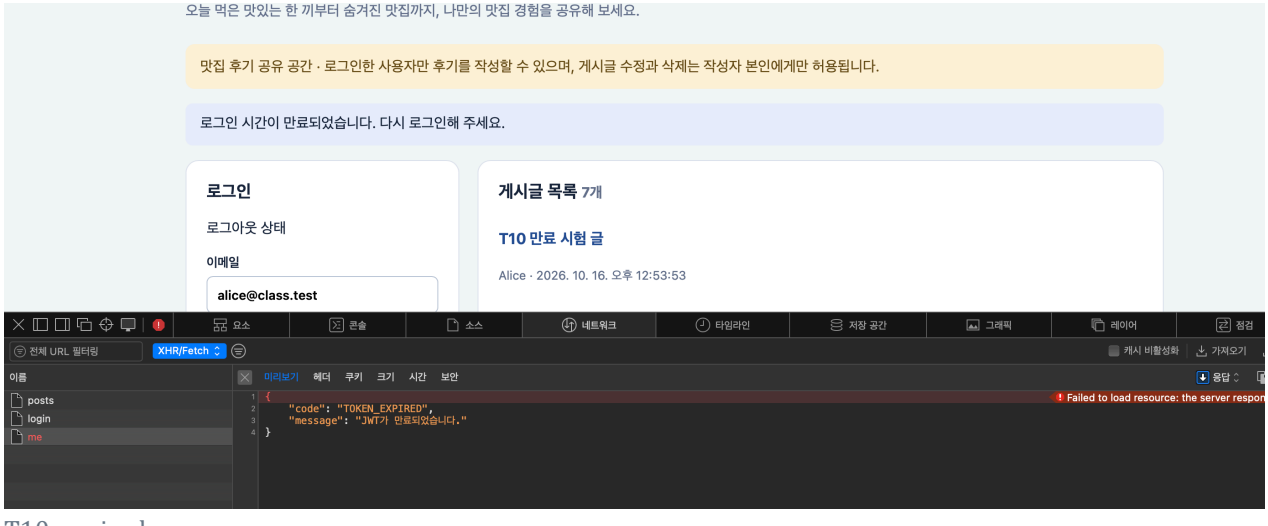
[P2(T09), P5, P4(T10), T10_expired-screen.png, T10_expired-headers.png, T10_expired-response.png]



T10_expired-screen.png



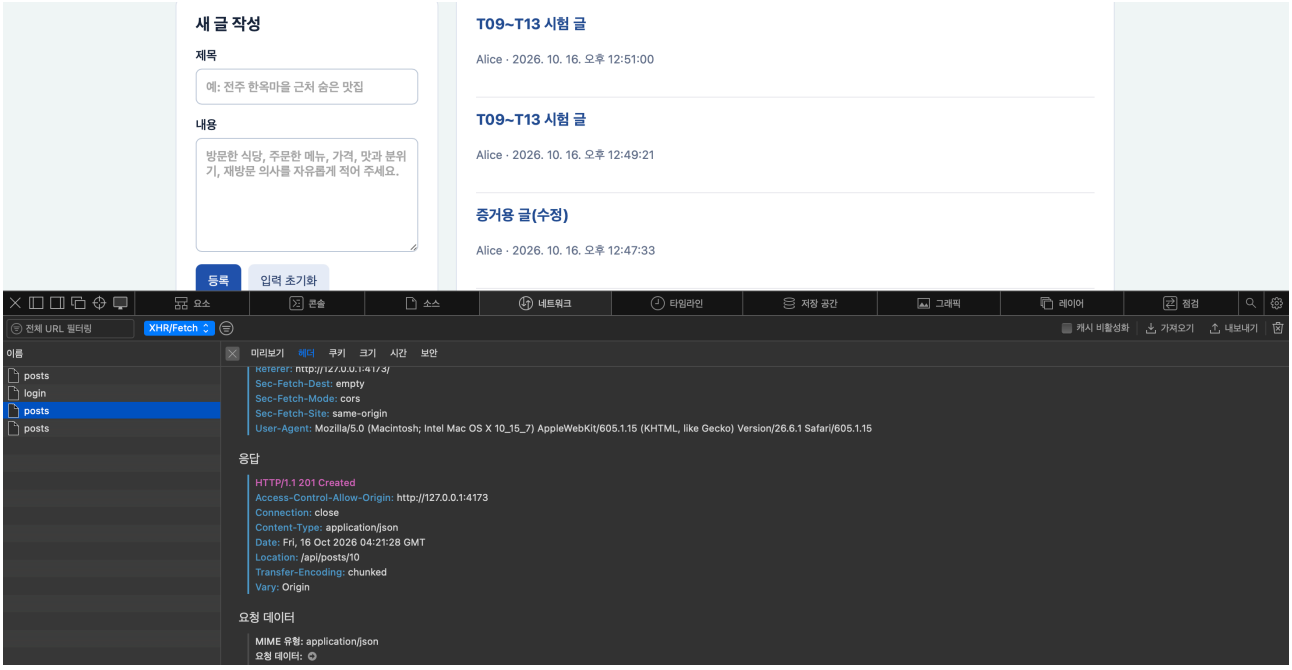
T10_expired-headers.png



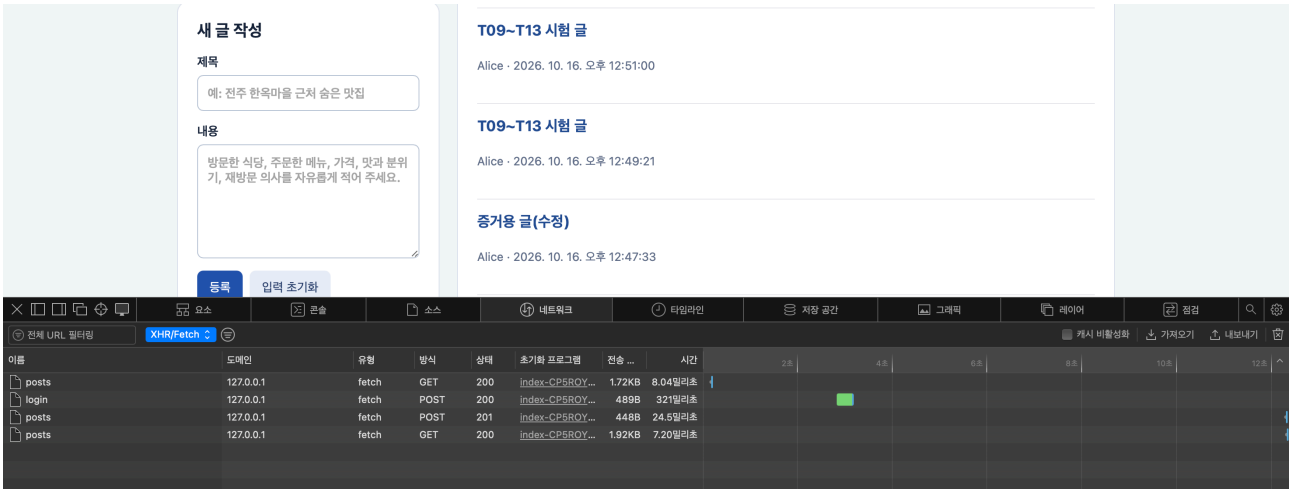
T10_expired-response.png

추가 테스트 증거

증거 4: 글 작성과 201 응답

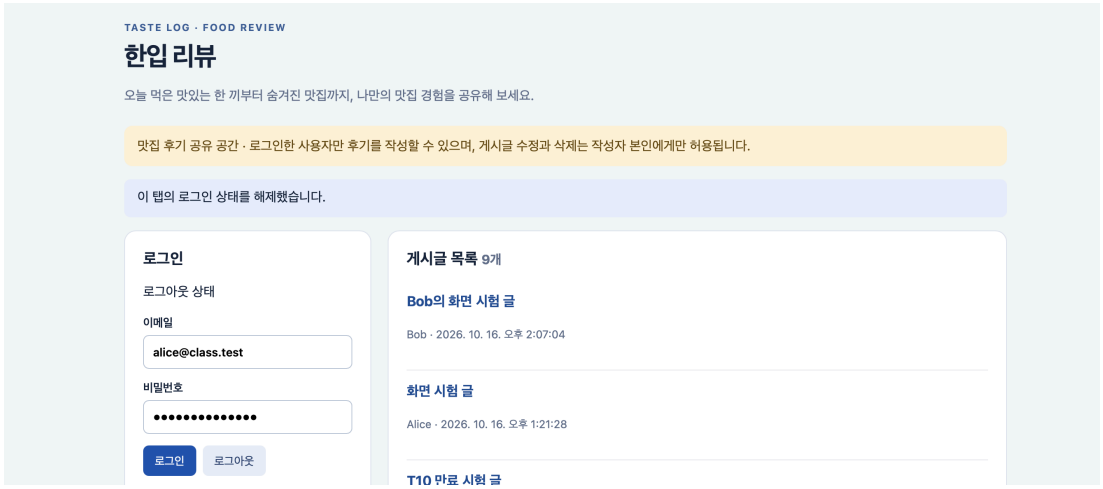


T05_location-header.png

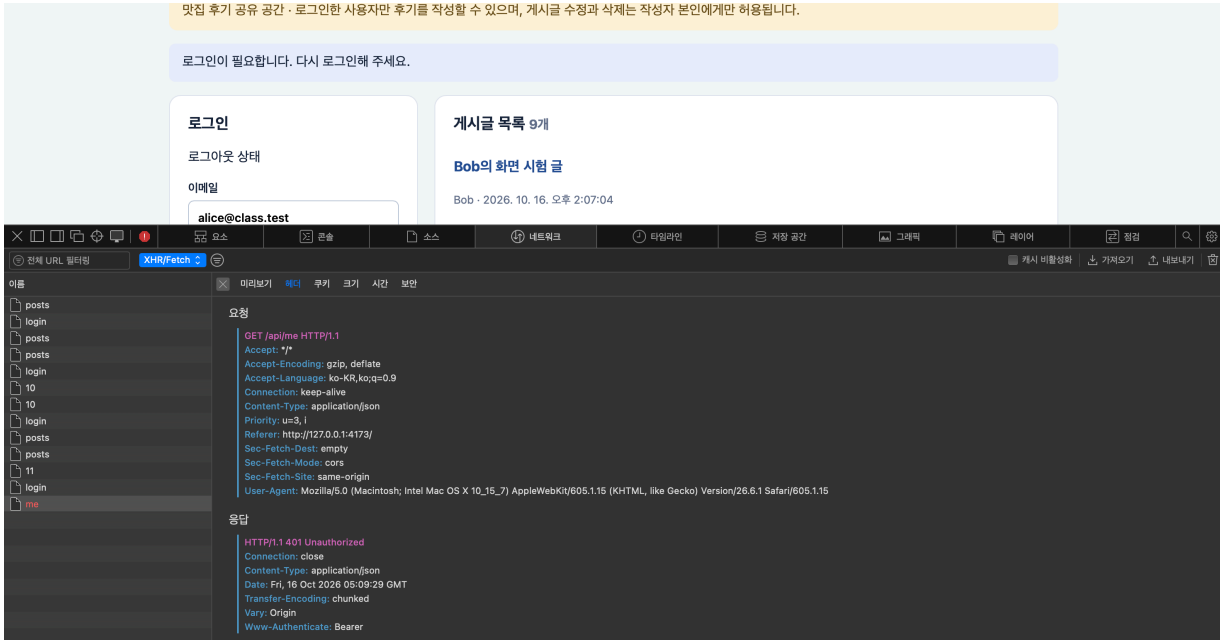


T05_post-created.png

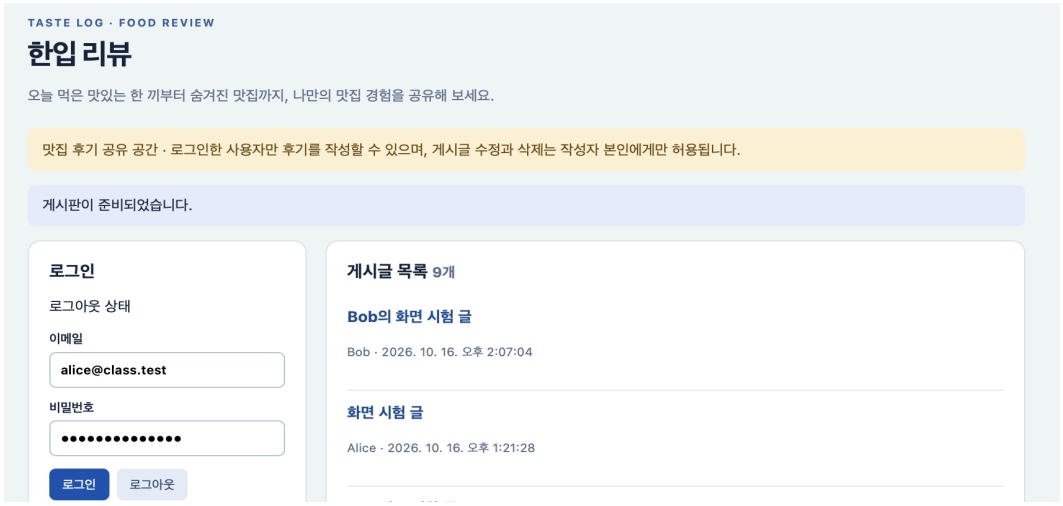
증거 5: 로그아웃과 새로고침



T11_logout.png

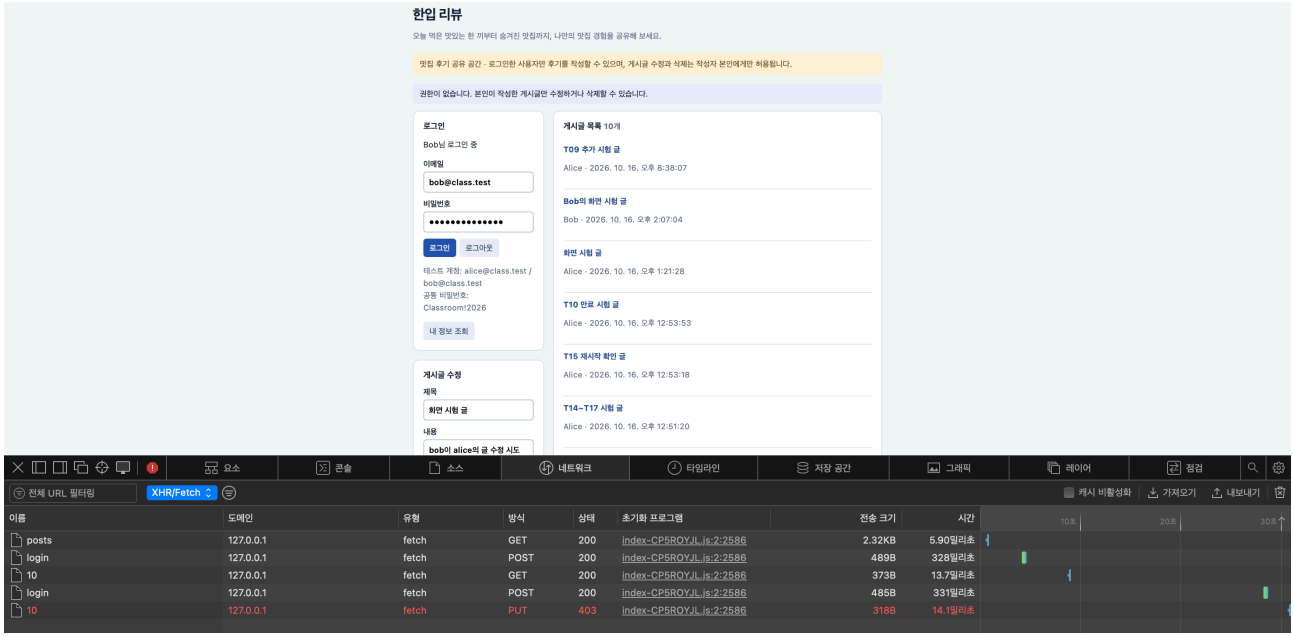


T11_no-auth-header.png



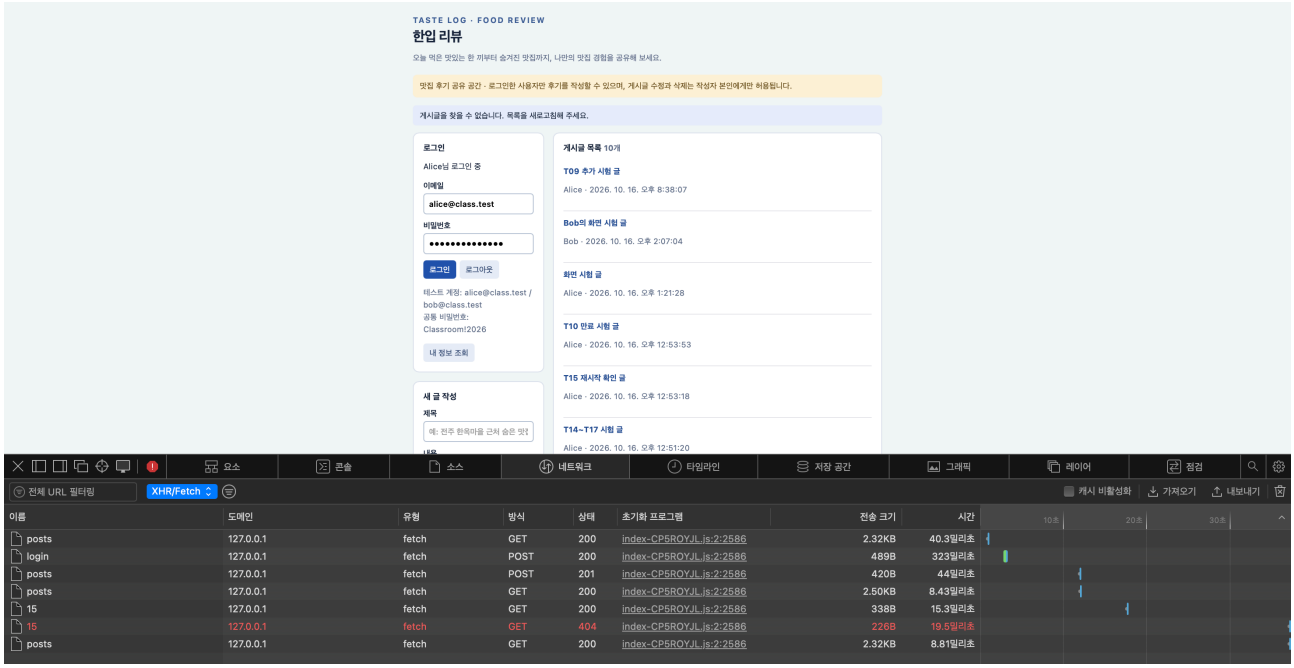
T16_after-refresh.png

증거 6: 403 화면안내 – Bob 으로 로그인한 상태에서 Alice 글의 수정 저장을 요청한 결과(상단 안내, 로그인 유지, PUT 403). 정상 조작에서는 타인 글에 수정 버튼이 보이지 않으므로, 로그아웃 없이 사용자를 바꿔 로그인하면 수정 폼이 남는 점을 이용해 요청을 보냈습니다.



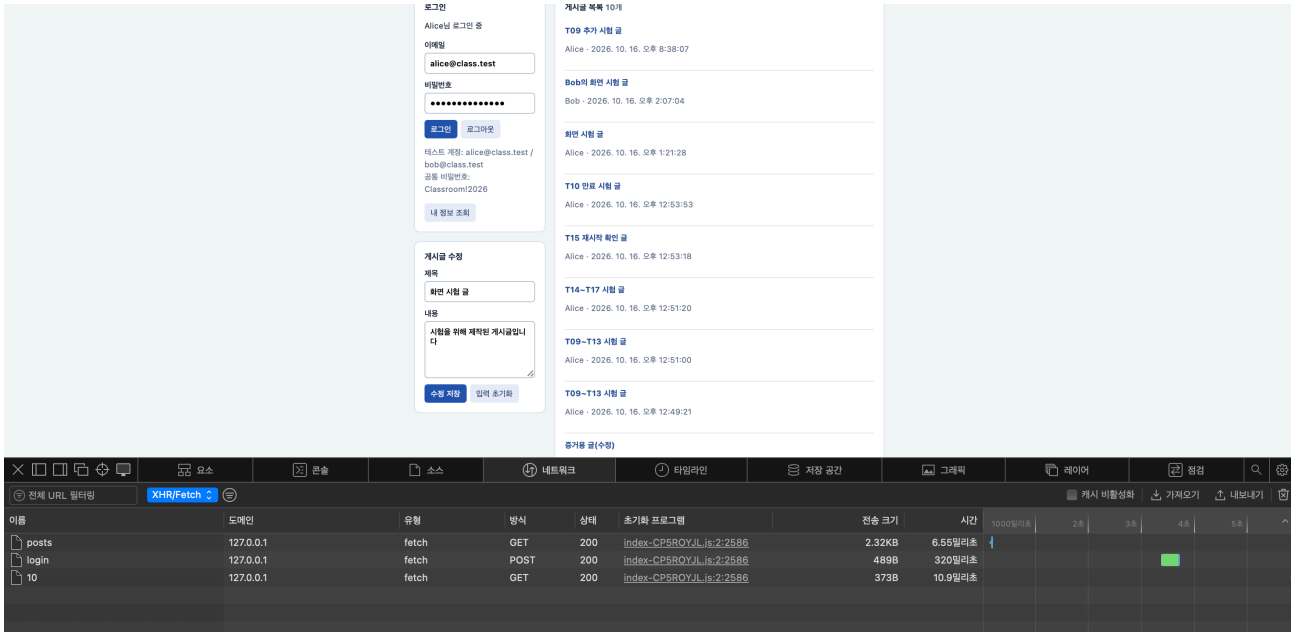
T07_403-screen.png

증거 7: 404 응답의 화면 동작 (FE03_404-screen.png) — Alice 로 404 시험 글을 등록해 상세를 열어 둔 뒤, 터미널의 curl 로 그 글을 삭제(DELETE, 204)했습니다. 브라우저를 새로고침하지 않고 목록에 남아 있는 같은 글을 누르자 상단에 "게시글을 찾을 수 없습니다. 목록을 새로고침해 주세요."가 표시되었고, 상세 영역이 "목록에서 제목을 선택하세요."로 돌아갔으며, 목록이 다시 불러와져 그 글이 사라졌습니다. 개발자 도구에는 posts/숫자의 404 요청과 목록 재조회(200)가 보였습니다.

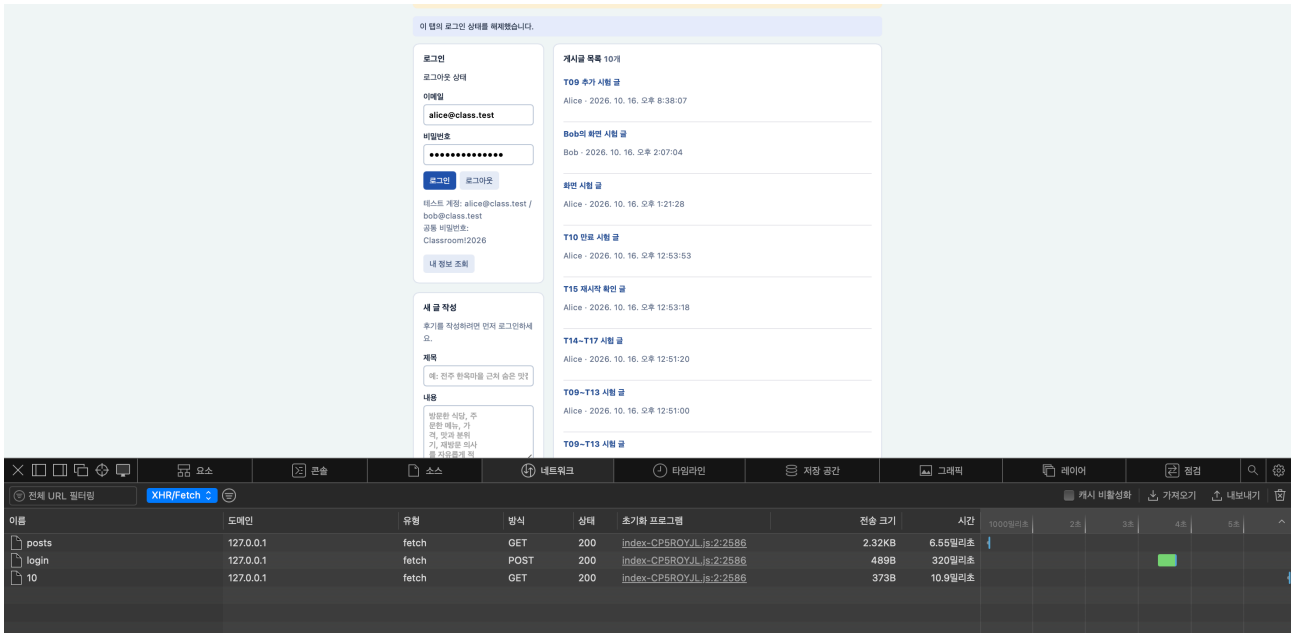


FE03_404-screen.png

증거 8: 로그아웃 시 입력 폼 초기화 (FE04_logout-form-before.png, FE04_logout-form-after.png) — 본인 글의 수정 폼을 연 상태(카드 제목 "게시글 수정", 제목·내용 채워짐)에서 로그아웃하자 입력 칸이 비워지고 카드 제목과 버튼이 "새 글 작성"/"등록"으로 돌아갔으며, 상단에 "이 탭의 로그인 상태를 해제했습니다."가 표시되고 로그인 카드가 "로그아웃 상태"가 되었습니다.

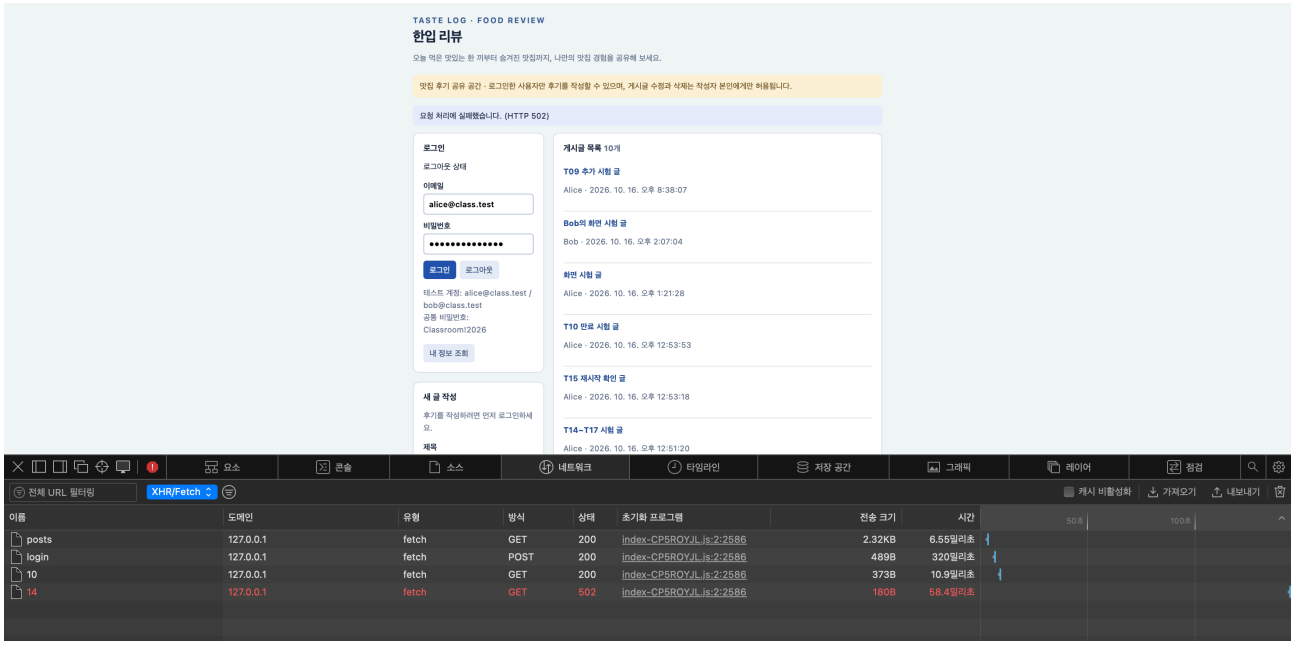


FE04_logout-form-before.png

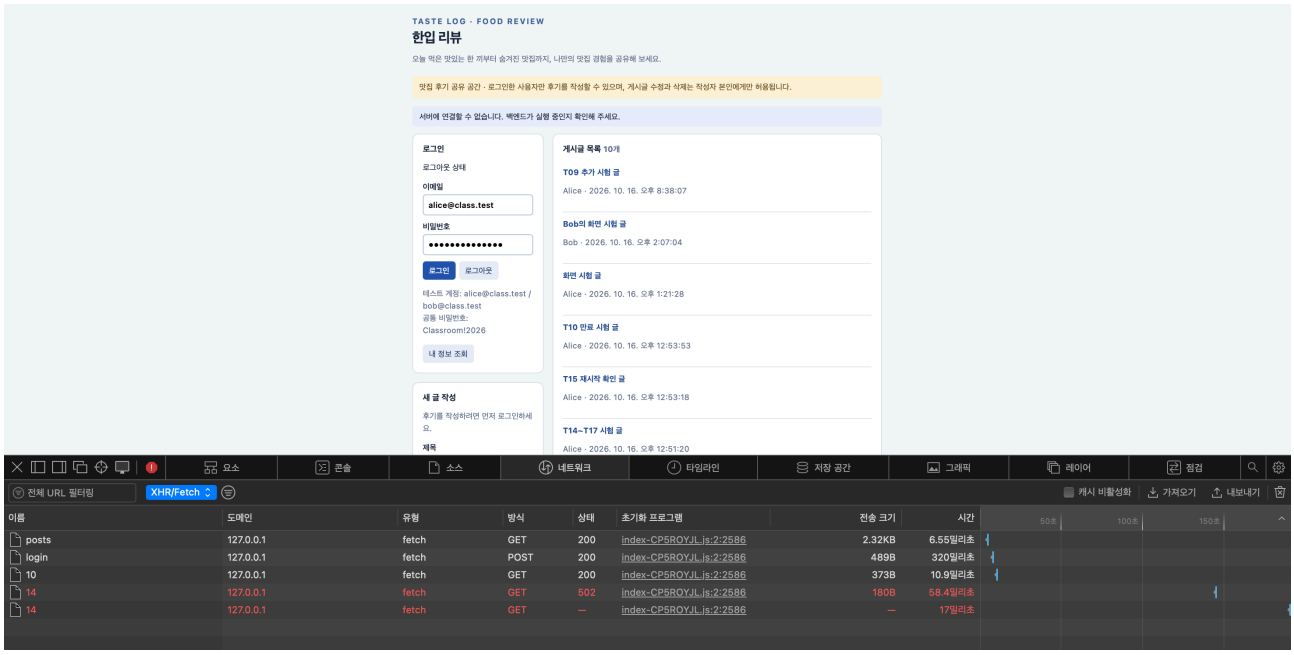


FE04_logout-form-after.png

증거 9: 서버가 꺼졌을 때의 안내 (FE03_backend-down.png, FE03_front-down.png) — 두 경우를 나누어 시험했습니다.

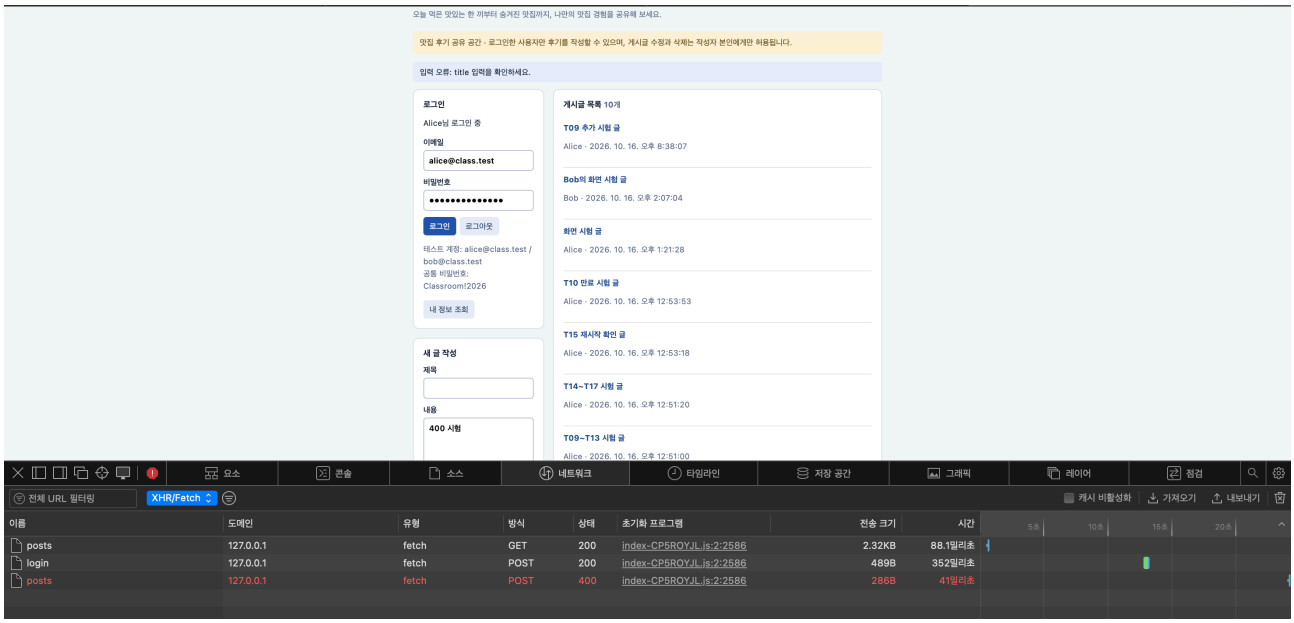


(FE03_backend-down.png)프론트 서버는 켜 둔 채 백엔드만 종료한 경우: 프론트 서버의 프록시가 JSON 이 아닌 502 응답을 돌려주었고, api.js 의 JSON 파싱 방어로 영어 예외 없이 "요청 처리에 실패했습니다. (HTTP 502)"가 표시되었습니다.



(FE03_front-down.png)프론트 서버까지 종료한 경우: fetch 자체가 실패해 "서버에 연결할 수 없습니다. 백엔드가 실행 중인지 확인해 주세요."가 표시되었습니다.

증거 10: 400 응답의 화면 동작 (FE03_400-screen.png) — Alice 로 로그인한 상태에서 제목 칸에 공백만 입력하고 내용 칸에 “400 시험”을 입력한 뒤 등록을 눌렀습니다. HTML 의 required 는 공백도 입력으로 보기 때문에 폼은 통과했고, 서버가 400 BAD_INPUT 을 돌려주었습니다. 상단에 "입력 오류: title 입력을 확인하세요."가 표시되었고, 내용 칸의 입력은 지워지지 않고 남았으며, 로그인은 유지되었고, 목록에 새 글이 생기지 않았습니다. 개발자 도구에서 POST 요청의 상태가 400 이었습니다.



FE03_400-screen.png

자동 검사와 로컬 배포 증거

백엔드를 켠 뒤 저장소 루트에서 python3 tools/check_api.py --mode final 을 실행해 40 개 항목이 모두 통과했습니다(api-check-final.txt). 이 검사는 일부 항목만 자동 확인하며 위 수동 시험을 대신하지 않습니다. --expiry 옵션의 자동 만료 검사는 이번 제출본에서 실행하지 않았고, 만료는 위 수동 시험(T10)으로 확인했습니다.

추가 시험(BE-01 키 검증): JWT_SECRET 을 설정하지 않은 경우, 예시 키 그대로, 빈 값, 16 진수 63 자리의 네 경우로 서버를 시작해 모두 시작이 거부되는 것을 확인했습니다(api-key-rejection.txt, 2026-10-18).

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: [사용]

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
[Codex(OpenAI)]	[코드 작성 제안]	[AuthService.java 의 생성자(키·설정 검증)와 validAudience, vite.config.js]	[제안 후 반영]
[Codex(OpenAI)]	[코드 초안]	[frontend-vue/src/api.js 의 getErrorMessage, canEdit]	[수정 후 반영]
[Claude(Anthropic)]	[과제 조건 대조 점검, 프론트 보완 제안]	[frontend-vue/src/api.js 의 request, frontend-vue/src/App.vue]	[그대로 반영]
[Claude(Anthropic)]	[테스트 명령과 증거 스크립트 작성]	[tools/manual/*.sh, tools/manual/mktoken.py, docs/evidence/]	[제안 명령을 실행하고 출력 오류를 고쳐 다시 실행]
[Claude(Anthropic)]	[README, backend-spring/README.md, frontend-vue/README.md 와 보고서 문장 초안]	[README.md, 보고서]	[초안 수정 후 반영]

대표 사례

사례 1: 연결 실패·JSON 파싱 실패 처리

도구 / 날짜: [Claude(Anthropic) / 2026-10-15]

요청한 내용: [과제 조건(FE-03)과 현재 프론트 코드를 대조해 부족한 점을 점검해 달라고 요청했고, 코드는 터미널 출력으로 붙여 넣어 제공했습니다.]

AI 가 제안한 내용: [fetch 호출과 response.json()을 각각 try/catch 로 감싸, 서버에 연결하지 못하거나 응답이 JSON 이 아닐 때 영어 예외 대신 한국어 안내가 나오게 하고, 본문이 없어도 data?.code 로 안전하게 처리하는 코드]

실제 반영 위치: [frontend-vue/src/api.js 의 request]

내가 확인하고 수정한 내용: [제안 코드를 거의 그대로 반영했고(들여쓰기만 다름), 기존 204 처리와 401 처리(clearSession)가 그대로 남아 있는지 sed 로 파일을 출력해 확인했습니다.]

검증 방법과 결과: npm run build 성공(빌드 결과 파일 해시가 바뀜), 화면 시험에서 400·401·403·404 처리와 만료 안내, 로그아웃이 정상 동작(T10, T11, 증거 6·7·10), 자동 검사 40 개 통과. 서버가 꺼진 경우도 직접

시험했습니다. 프론트 서버를 켜 둔 채 백엔드만 종료하면 프록시가 JSON 이 아닌 502 응답을 돌려주어 파싱 방어로 "요청 처리에 실패했습니다. (HTTP 502)"가 표시되었고 영어 예외는 나타나지 않았습니다. 프론트 서버까지 종료하면 fetch 가 실패해 "서버에 연결할 수 없습니다. 백엔드가 실행 중인지 확인해 주세요."가 표시되었습니다(증거 9).

사례 2: 404 처리·로그아웃 폼 초기화·비로그인 안내

도구 / 날짜: [Claude(Anthropic) / 2026-10-15]

요청한 내용: [과제 조건 FE-03(404 이면 목록 갱신)과 FE-04(로그아웃 시 화면 표시 지우기)를 충족하는지 점검]

AI 가 제안한 내용: [run()의 catch 에서 404 이면 선택과 폼을 지우고 목록을 다시 불러오기, logout()에 reset() 추가, 새 글 작성 카드에 비로그인 안내 문구]

실제 반영 위치: [frontend-vue/src/App.vue]

내가 확인하고 수정한 내용: [처음 제안은 run 함수의 catch 부분만 바꾸라는 설명이었는데 범위가 모호해서, 적용하다가 run 함수의 앞부분이 지워져 문법이 깨졌습니다. 파일 출력으로 확인한 뒤 Claude 가 준 전체 코드로 파일을 통째로 교체했고, grep 으로 반영 위치를 확인한 다음 npm run build 로 성공을 확인했습니다.]

검증 방법과 결과: 빌드 성공, 비로그인 안내 문구가 화면에 표시되는 것을 확인했습니다. 404 는 다른 경로로 글을 삭제한 뒤 목록에 남은 글을 다시 누르면 "게시글을 찾을 수 없습니다. 목록을 새로고침해 주세요."가 표시되고 상세 선택이 해제되며 목록에서 그 글이 사라졌습니다(증거 7). 로그아웃은 수정 폼을 연 상태에서 누르면 입력 칸이 비워지고 "새 글 작성"/"등록"으로 돌아갔습니다(증거 8). 로그아웃 후 상태와 새로고침 시험(T11, T16)도 통과했습니다.

사례 3: 테스트 명령과 증거 스크립트 추가

도구 / 날짜: [Claude(Anthropic) / 2026-10-16]

요청한 내용: [교수님 시나리오(T01~T17)를 증거가 남는 방식으로 실행하는 명령, 그리고 JWT 조작 시험]

AI 가 제안한 내용: [curl 로 요청과 응답을 기록하고 토큰을 <TOKEN>으로 가리는 스크립트(part1~4.sh), 서명은 올바르고 내용만 바꾼 토큰을 시험용 키로 만드는 mktoken.py 와 시험 스크립트(part5.sh)]

실제 반영 위치: [tools/manual/, docs/evidence/]

내가 확인하고 수정한 내용: [제안 명령을 직접 실행해 응답을 하나씩 예상과 대조했습니다. 실행 중 zsh 의 ! 처리 오류와 출력 문구의 변수 오류($N0 과)가 있어 수정한 명령으로 다시 실행했고, 최종 증거는 다시 실행한 결과입니다. 시험용 키는 저장소 밖의 임시 파일에 두었다가 시험 뒤 삭제했습니다.]

검증 방법과 결과: [대조군(직접 만든 정상 토큰)이 200 으로 통과해 토큰 생성 방식이 맞음을 확인했고, 조작 토큰 31 건 중 FAIL 은 없었습니다. 증거 파일에서 실제 토큰 문자열이 남지 않았는지 검색했습니다.]

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
[수업 6 주차·7 주차 강의 자료(키 설정 부분)]	[AuthService 생성자]	[서버 키로 발급·검증하는 구조를 참고하고 Codex 제안을 함께 반영]
[수업 6 주차 “JWT 로그인” 28 쪽]	[AuthService.issueToken]	[토큰 발급 구조를 참고]

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
[수업 6 주차 35~40 쪽, 7 주차 “JWT 접근 제어와 Swagger” 50·53 쪽]	[AuthService.currentUserId]	[Bearer 읽기와 토큰 검증 구조를 참고]
[수업 7 주차 65 쪽]	[AuthService.assertOwner]	[작성자 비교 논리를 참고]
[수업 6 주차 31·32·47 쪽]	[frontend-vue/src/api.js]	[토큰 메모리 보관, 헤더 첨부, 401 처리 구조를 참고]
[과제 제공 템플릿(JWT-Assignment-2), docs/02_API 명세.md]	[전체]	[제공 코드 유지 부분과 API 계약]

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

[요청 본문의 authorId 를 믿으면 누구나 다른 사람 이름으로 글을 쓸 수 있습니다. 예를 들어 Bob 이 {"authorId":1}을 붙여 보내면 Alice 가 쓴 글로 저장됩니다. 내 코드는 작성자를 요청에서 받지 않고, BoardController.create 에서 AuthService.currentUserId 가 토큰을 검증해 돌려준 사용자 ID 를 서버가 직접 저장합니다. 또 fields 가 title 과 content 두 필드만 허용해서 authorId, userId, id 가 들어 있으면 400 BAD_INPUT 으로 거부합니다. 수정 SQL(PostStore.update)도 title, content, updated_at 만 바꾸므로 작성자와 createdAt 은 바뀌지 않습니다. 시험(T12)에서 authorId, userId, id 를 붙인 등록 3 건과 수정 1 건이 모두 400 이었고 글 개수와 원문이 그대로였습니다.]

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

[버튼 숨김은 화면만 바꾸는 것이라서, 개발자 도구나 curl 로 서버에 직접 요청을 보내면 막을 수 없습니다. 그래서 서버의 AuthService.assertOwner 가 현재 사용자 ID 와 글에 저장된 authorId 를 비교해, 다르면 403 FORBIDDEN 을 던집니다. BoardController.update 와 delete 가 글 존재 확인(404) 뒤에 이 함수를 호출하고, 통과하지 못하면 입력 검사와 DB 변경 단계에 도달하지 않습니다. 프론트의 canEdit 은 사용자가 할 수 없는 작업을 눈에 띄지 않게 하는 편의 기능입니다. Bob 의 토큰으로 Alice 글을 직접 수정·삭제했을 때 403 이었고 글이 그대로였으며(T07, T08), 화면의 수정 폼으로 보낸 요청도 403 이었습니다(증거 6).]

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

[decode 는 토큰 안의 내용을 Base64 로 풀어서 읽기만 하고 서명을 확인하지 않습니다. 그래서 누군가 sub 를 바꿔도 알 수 없습니다. 실제로 발급된 토큰의 내용을 직접 풀어서 읽을 수 있었습니다(jwt-claims.txt). verify 는 서버의 비밀 키로 서명을 확인하고 만료도 검사하므로, 내용이 바뀌면 거부됩니다. 내 코드는 Jwts.parser().verifyWith(key).build().parseSignedClaims(token)으로 서명과 만료를 검증한 뒤, 알고리즘이 HS256 인지, exp 가 있는지, iss 와 aud 가 서버 설정과 같은지, sub 가 양의 정수 문자열인지, 그 사용자가 실제로 있는지를 추가로 확인합니다. 서명 첫 글자를 바꾼 토큰 4 건은 모두 401 INVALID_TOKEN 이었고(T09), 서명은 맞고 내용만 바꾼 조작 토큰 31 건도 예상대로 거부되었습니다(P5).]

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

[로그아웃은 브라우저 메모리의 토큰과 사용자를 비우고(clearSession) 화면의 로그인 표시와 입력 폼도 지웁니다. 이후 요청에는 Authorization 헤더가 붙지 않아 서버가 401 TOKEN_REQUIRED 로 응답합니다(T11). 토큰 만료는 토큰의 exp 가 지나면 서버가 401 TOKEN_EXPIRED 로 거부하고, 화면은 401 을 받아 토큰과 사용자를 해제한 뒤 만료 안내를 보여 줍니다(T10). 새로고침은 토큰을 저장한 자바스크립트 변수가 사라져 로그인 상태가 해제됩니다. 게시글은 DB 에 있어서 그대로 남습니다(T16)

이미 복사된 토큰은 로그아웃해도 서버가 토큰을 기억하거나 무효화하지 않으므로 만료 시각(기본 90 초)까지 유효합니다. 로그아웃은 이 탭의 상태만 지우는 기능이며, 서버에서 토큰을 즉시 철회하려면 별도 설계가 필요합니다]

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

[App.vue 를 수정하던 중 파일이 깨진 문제입니다. Claude 가 제안한 run 함수의 catch 블록을 적용하다가, run 함수의 앞부분(async function run, try 블록)이 같이 지워져서 } catch (e) {로 시작하는 문법이 맞지 않는 코드가 되었습니다. 파일의 첫 부분을 sed 로 출력해 refresh 바로 아래에 catch 만 남아 있는 것을 보고 원인을 확인했습니다. 설명한 교체 범위가 모호해서 한 블록을 통째로 덮어쓴 것이 원인이었습니다. 부분 수정 대신 완성된 App.vue 전체를 받아 파일을 통째로 교체했고, grep 으로 reset();이 네 곳(404 처리, 저장, 삭제, 로그아웃)과 로그인 안내 문구에 들어갔는지 확인한 뒤 npm run build 가 오류 없이 끝나는 것으로 재검증했습니다. 이후 화면에서 404 처리와 로그아웃 폼 초기화를 직접 시험했습니다(증거 7, 8).]

현재 남은 문제와 다음에 개선할 점: [로그인할 때 입력 폼이 초기화되지 않아, 사용자를 바꿔 로그인하면 이전 사용자가 열어 둔 수정 폼이 남습니다. 그 폼으로 저장해도 서버가 403 으로 거부해 데이터에는 영향이 없지만(증거 6), 로그인 시에도 폼을 초기화하도록 개선할 수 있습니다.

프론트 서버는 켜져 있고 백엔드만 꺼진 경우, 화면에는 "요청 처리에 실패했습니다. (HTTP 502)"처럼 일반적인 안내만 표시됩니다. 502 도 "서버에 연결할 수 없습니다" 안내로 연결하면 원인을 더 쉽게 알 수 있습니다.

동시에 두 사용자가 같은 글을 수정하는 상황은 시험하지 않았습니다.]

제출 확인: o 실제 코드와 설명 일치 o 테스트 직접 수행 o AI 활용 및 출처 기록 o 비밀 키와 토큰 제거 o

GitHub 확인: o 루트 README 작성 o 보고서 PDF 와 증거 업로드 o README 문서 링크 확인 0 교수자 접근 가능 o 저장소 URL 과 커밋 SHA 제출 o